



Multimedia Retrieval System

Prof. dr. Alexandru C. Telea

Students: Binxu Jiang (1524313), Amjad Hwidy (2465183)

Date: 14-11-2025 | Utrecht University

Table of Contents

1. Introduction.....	1
2. Implementation Choices.....	1
2.1. General Notation.....	1
3. Step 1 - Reading and Viewing the Data	4
3.1. Aim	4
3.2. Method.....	4
3.3. Result.....	4
4. Step 2 – Preprocessing and Cleaning.....	8
4.1. Aim	8
4.2. Method.....	8
4.2.1. Step 2.1 – Analysing a single shape.....	8
4.2.2. Step 2.2 – Statistics over the whole database.....	9
4.2.3. Step 2.3 – Resampling outliers	10
4.2.4. Step 2.4 – Checking the resampling	11
4.2.5. Step 2.5 – Normalization.....	11
4.3. Results.....	13
4.3.1. Translation Verification	13
4.3.2. Scaling Verification	13
4.3.3. Topology and Resolution Consistency.....	14
4.3.4. Representative Mesh Comparisons	14
4.4. Conclusion	16
5. Step 3 - Feature Extraction.....	17
5.1 Aim	17
5.2 Normalization	17
5.3 3D shape descriptors.....	20
5.4 Results.....	23
6. Step 4 - Query	33
6.1. Feature normalization-based query	34
6.2. Query improvement	37
6. Step 5: Scalability.....	42

6.1.	Data Preparation and Feature Loading	42
6.2.	Efficient Neighbour Search.....	42
6.2.1.	Exact k-Nearest Neighbours	42
6.2.2.	Approximate Nearest Neighbours (ANN).....	42
6.3.	Feature Space Visualization with t-SNE	43
6.4.	Interactive Retrieval Interface	44
6.5.	Conclusion of Step 5.....	45
7.	Step 6: Evaluation	46
7.1.	Methodology	46
7.2.	Quantitative Results.....	46
7.2.1.	Precision–Recall Analysis for All Normalized Classes.....	46
7.2.2.	Precision–Recall Analysis for 10 Cleaned Classes	47
7.2.3.	Sensitivity–Specificity Analysis.....	48
7.2.4.	Retrieval Outcome Distribution.....	50
7.3.	Discussion of Results	51
7.4.	Conclusion of Step 6.....	51
8.	Summary	52

1.Introduction

The term ‘multimedia retrieval’ can be traced back to the idea of information retrieval, defined by IBM as “finding unstructured data within a large collection in order to satisfy a particular information need.” (Murel and Syed). Multimedia retrieval is an expansion of this field, instead of unstructured data it can be one of many different formats of media e.g. images, videos, 3D objects, etc. This report chooses to focus on mainly the 3D object side of multimedia.

2.Implementation Choices

Python was chosen as the programming language for this project. The main reasons are that it is easy to learn and write, has many useful libraries for data processing and visualization, and is widely used in both research and industry. These advantages make it a practical choice for building and testing multimedia retrieval tools.

2.1. General Notation

To avoid repeating formulas throughout the report, all mathematical notation used is summarized here.

1. Mesh Representation

A 3D object is represented as a mesh $M = (V, F)$

Where:

- $V = \{v_i \in \mathbb{R}^3 \mid i = 1, \dots, N_V\}$ is the set of vertices, and
- $F = \{f_j\}$ is the set of triangular faces.

The cardinalities $|V| = N_V, |F| = N_F$ denote the number of vertices and faces.

2. Axis-Aligned Bounding Box (AABB)

The bounding box is defined by the coordinate-wise minimum and maximum of all vertices: $v_{\min} = \min(V), v_{\max} = \max(V)$.

The **extent** of the bounding box is $e = v_{\max} - v_{\min}$.

The **bounding-box diagonal** (overall scale indicator) is $d_{\text{bbox}} = \|e\|_2$.

3. Centroid

The geometric centroid (barycenter) of the mesh is $c = \frac{1}{N_V} \sum_{i=1}^{N_V} v_i$.

4. Normalization (Centering and Scaling)

Normalization consists of two steps:

- **Centering** the mesh at the origin: $v'_i = v_i - c$.
- **Uniform scaling** so the largest side of the bounding box becomes 1: $s = \frac{1}{\max(e)}, v''_i = s v'_i$.

Normalization preserves topology: $|V''| = |V|, |F| = |F|$.

5. Face-to-Vertex Ratio

During simplification, the ratio $R = \frac{N_F}{N_V}$

is used to estimate a stable target number of triangles.

6. Normalization Checks

Two scalar checks verify that normalization was applied correctly:

- **Centroid after normalization:** $\|c'\| \approx 0$.
- **Maximum extent after scaling:** $\max(e') \approx 1$.

These conditions confirm that every mesh is properly centered and scaled inside the unit cube.

7. Compactness

We used the following equation to calculate the compactness, which measures how similar the shape is to a sphere.

$$C = \frac{S^3}{36\pi V^2}$$

Where S represents surface area and V represents volume.

8. 3D rectangularity

We used the following equation to calculate the 3D rectangularity:

$$R = \frac{V}{V_{OOB}}$$

V is the calculated volume, V_{OOB} is the volume of Oriented Bounding Box.

9. Convexity

Convexity is computed by:

$$c = \frac{V}{V_{hull}}$$

Where V is the shape volume, V_{hull} is the convex hull volume.

10. Eccentricity

Eccentricity is the ratio of the largest to the smallest eigenvalues of the covariance matrix:

$$E = \frac{|\lambda_1|}{|\lambda_3|}$$

Where λ_1 is the largest eigenvalue and λ_3 is the smallest eigenvalue of the covariance matrix.

11. Features distance

Let $A = (a_1, \dots, a_n)$ be a feature vector. Typically A contains elements that describe different features, each having its own value. We have a feature vector of 507 elements. The first seven elements ($a_1 \dots a_7$) indicate elementary features, e.g., surface area, volume, compactness, rectangularity, diameter, convexity and eccentricity. Next, we have 100-bin histograms for the distribution descriptors A3 ($a_8, \dots, a_{107}$), D1 ($a_{108}, \dots, a_{207}$), D2 ($a_{208}, \dots, a_{307}$), and D3 ($a_{308}, \dots, a_{407}$), and D4 ($a_{408}, \dots, a_{507}$).

For the seven single-value descriptors($a_1, \dots, a_7$), we computed the feature distance dimension-wise using the L1 metric:

$$d_{elementary, i} = |a_i^{(q)} - a_i^{(m)}|, i = 1, \dots, 7$$

Where $a_i^{(q)}$ and $a_i^{(m)}$ denote the i -th feature of the query mesh and a database mesh, respectively.

Each of these distances is scaled by a weight w_i , producing the weighted single-value distance:

$$d_{SV} = \sum_{i=1}^7 w_i d_{elementary, i}$$

For the five distribution descriptors, we used the Earth Mover's Distance (EMD), implemented using the 1D Wasserstein distance, to measure the difference between two histograms:

$$d_{EMD, k} = W_1(H_k^{(q)}, H_k^{(m)}), k = 1, \dots, 5$$

where H_k denotes the k -th 100-bin distribution (A3, D1, D2, D3, D4). Each histogram distance is also weighted:

$$d_{HD} = \sum_{k=1}^5 \alpha_k d_{EMD, k}$$

Finally, the total similarity distance is computed by:

$$d_{total} = d_{SV} + d_{HD}$$

3.Step 1 - Reading and Viewing the Data

3.1. Aim

The goal of this step was to implement a minimal application able to load and display 3D meshes from the provided database, and to allow basic interactive inspection of the shapes.

3.2. Method

We used **Python 3.11** with the **Open3D 0.190** and **NumPy 2.3.3** libraries to build a simple viewer. The program loads a mesh, computes vertex normals, and provides multiple different display modes: smooth shaded, flat shaded, and unshaded, each with a toggleable wireframe, and the wireframe by itself. Interactive operations (rotate, zoom, pan) are supported through the Open3D window.

The 3D meshes were obtained from the **ShapeDatabase_INFOMR** repository and organized locally in the **data** folder by object class. A virtual environment was used to manage dependencies; installation details are provided in the project's README.

The mesh is read by the `read_triangle_mesh` function from Open3D, which is stored in the variable `mesh`. A wireframe can be generated from this, using the `LineSet.create_from_triangle_mesh` function, which is stored in the variable `wireframe`. `EdgesDrawn` is a Boolean variable that controls two display modes: displaying only the mesh or displaying both the mesh and the wireframe. For smooth shading, the function `mesh.compute_vertex_normals()` is used, whereas for flat shading the function `mesh.compute_triangle_normals()` is used. The final mesh is visualized using the `o3d.visualization.draw` function, which can be called with or without the wireframe for different viewing options. For example, a smooth shaded object is visualized so:

```
mesh.compute_vertex_normals()
if edgesDrawn:
    o3d.visualization.draw([mesh, wireframe], width=1280, height=720,
                           show_skybox=False)
else:
    o3d.visualization.draw([mesh], width=1280, height=720, show_skybox=False)
```

3.3. Result

The viewer successfully loaded multiple objects from the dataset, confirming compatibility with meshes of different sizes and qualities. The multiple visualization methods were

tested alongside the pan, zoom, and rotate tools. Furthermore, different objects were analyzed to get a greater understanding of the dataset:

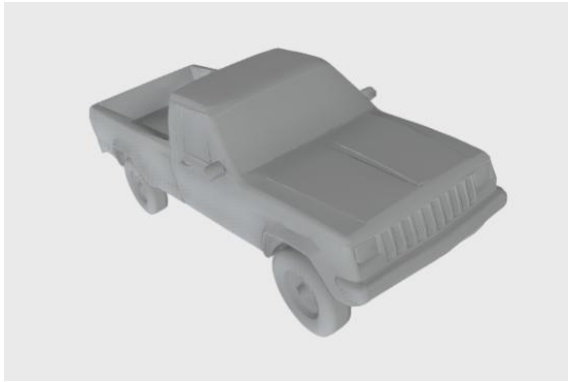


Figure 1: Car model displayed with smooth shading.



Figure 2: : Car model displayed with flat shading.

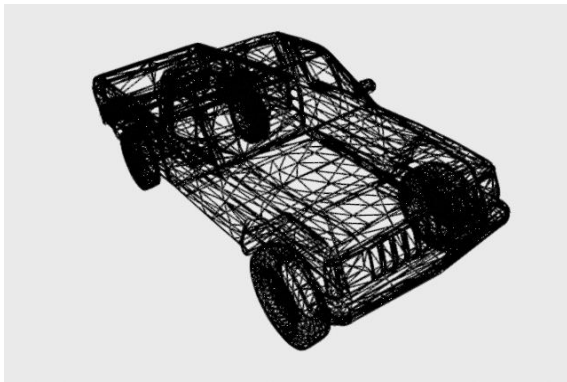


Figure 1: Car model displayed with a wireframe.

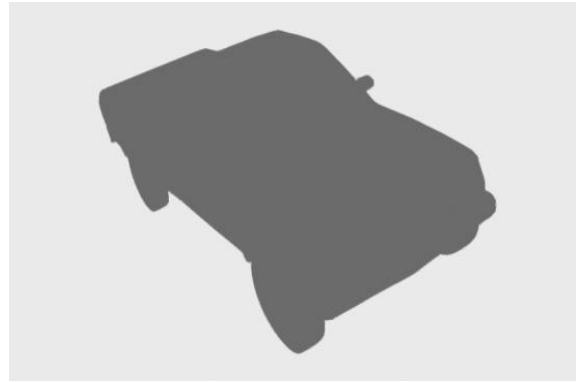


Figure 2: Car model displayed without any shading



Figure 4: Car model displayed with a wireframe overlay on an unshaded mesh

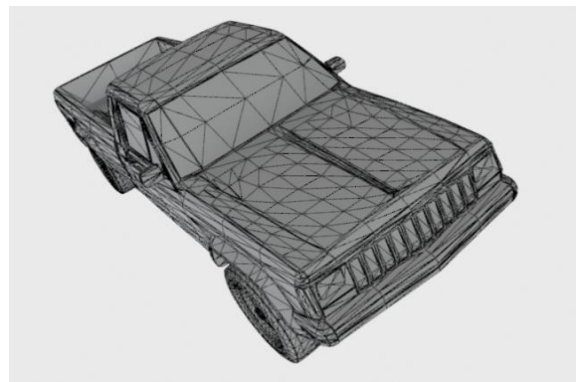


Figure 3: Car model displayed with a wireframe overlay on a smooth shaded mesh

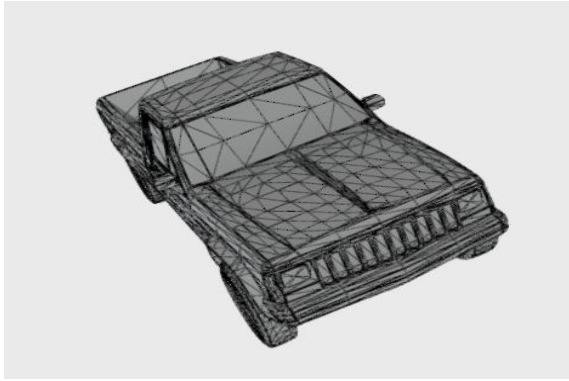


Figure 6: Car model displayed with a wireframe overlay on a flat shaded mesh

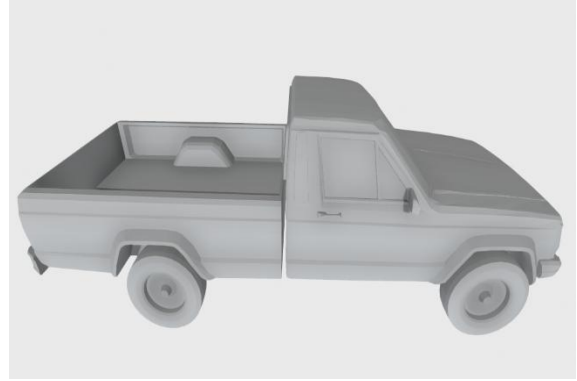


Figure 5: A rotated, zoomed in angle of the car model with flat shading.

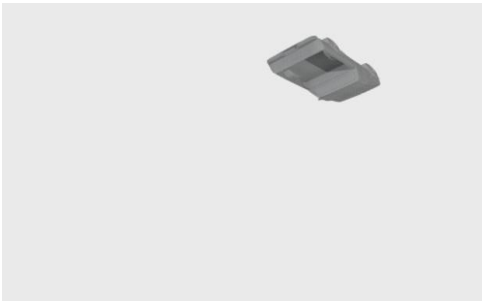


Figure 8: A rotated, zoomed in angle of the car model with flat shading.

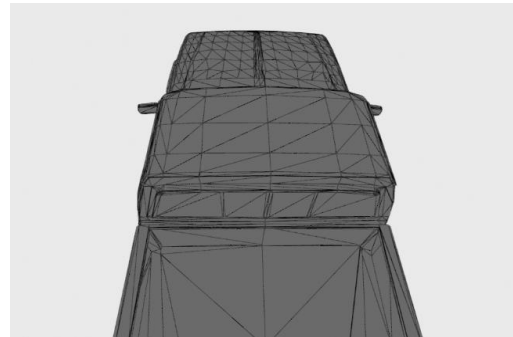


Figure 7: A rotated, zoomed in, panned angle of the car model with a wireframe overlay on an unshaded mesh.



Figure 9: A starship model with smooth shading. 1704 vertices, 3384 triangles.

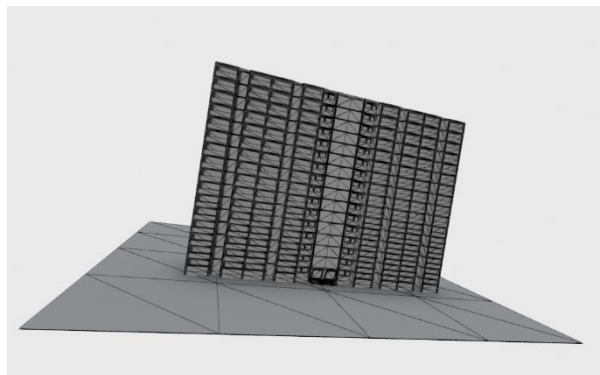


Figure 10: An apartment model with a wireframe overlay on a flat shaded mesh. 14865 vertices, 21021 triangles.

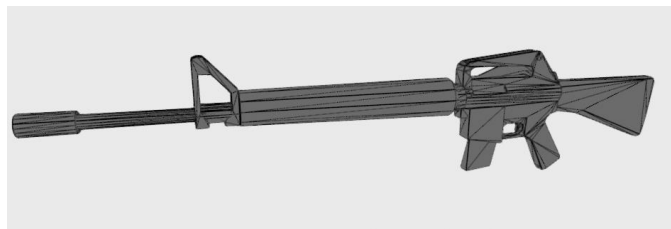


Figure 11: A gun model with a wireframe overlay on an unshaded mesh. 451 vertices, 785 edges.

Figures 1 through 4 show the different visualization methods that are possible in the viewer, those being smooth shading, flat shading, unshaded, and a wireframe. Moreover, Figures 5 through 7 show the wireframe overlay which is a toggleable option. Figures 8-10 display the car model from different angles using the pan, zoom, and rotate features. Finally, Figures 11-13 view different objects to analyze the various forms of mesh that exist in the dataset. Analyzing the results, the possible ranges of the number of vertices appears to be very large, going from the 14865 vertices of the apartment in Figure 12 to the 451 vertices of the gun in Figure 13.

4. Step 2 – Preprocessing and Cleaning

4.1. Aim

This step prepares the 3D database for reliable feature extraction and similarity retrieval. The raw meshes vary enormously in resolution, scale, and structure: some objects contain only a few dozen vertices, while others exceed sixty thousand. Without preprocessing, feature descriptors would reflect these inconsistencies instead of capturing genuine 3D geometry.

The goal of Step 2 is therefore to convert the raw database into a **uniform, high-quality collection of meshes**. This involves (1) inspecting the structure of each object, (2) examining global statistics across the dataset, (3) resampling meshes that are too coarse or too dense, and (4) normalizing every object so they share the same coordinate frame and scale.

By the end of this step, all meshes should have comparable resolution, be centered at the origin, and fit inside the same bounding-box size, making the dataset suitable for later feature extraction.

4.2. Method

All the preprocessing and cleaning was done in **Python 3.11** using the **Open3D 0.19.0**, **NumPy 2.3.3**, **Pandas 2.3.3**, and **Matplotlib 3.10.6** libraries.

4.2.1. Step 2.1 – Analysing a single shape

The first step was to examine each mesh individually. For every object, the program recorded its number of vertices and faces, the type of faces (triangles, quads, or mixed), and its bounding-box dimensions. Class labels were inferred automatically from the folder structure.

This inspection served two purposes. First, it verified that all meshes were readable, non-empty, and consistently formatted. Second, it provided a clear picture of the variation within the dataset.

A short sample of ten meshes (Figure 12) showed exactly the kind of extremes that motivated resampling: some models had fewer than thirty vertices, while others had several thousand faces and large bounding boxes. These differences are too large to be ignored in a retrieval system.

	cls	path	ext	n_vertices	n_faces	n_triangles	face_type	bbox_min_x	bbox_min_y	bbox_min_z	bbox_max_x	bbox_max_y	bbox_max_z	extent_x	extent_y	extent_z	bbox_diag
0	AircraftBuoyant	data\AircraftBuoyant\ym1337.obj	.obj	201	340	340	triangles	0.025	0.025	0.025	0.889234	0.975000	0.889234	0.864234	0.950000	0.864234	1.547999
1	AircraftBuoyant	data\AircraftBuoyant\ym1338.obj	.obj	8761	16960	16960	triangles	0.025	0.025	0.025	0.933058	0.975000	0.933058	0.908058	0.950000	0.912901	1.600143
2	AircraftBuoyant	data\AircraftBuoyant\ym1339.obj	.obj	1878	2790	2790	triangles	0.025	0.025	0.025	0.607072	0.975000	0.626812	0.582072	0.950000	0.601812	1.266288
3	AircraftBuoyant	data\AircraftBuoyant\ym1340.obj	.obj	778	1454	1454	triangles	0.025	0.025	0.025	0.367686	0.975000	0.367689	0.342686	0.950000	0.342689	1.066475
4	AircraftBuoyant	data\AircraftBuoyant\ym1341.obj	.obj	424	824	824	triangles	0.025	0.025	0.025	0.379575	0.975000	0.379575	0.354575	0.950000	0.354575	1.074219
5	AircraftBuoyant	data\AircraftBuoyant\ym1342.obj	.obj	1252	2302	2302	triangles	0.025	0.025	0.025	0.611587	0.975000	0.609360	0.586587	0.950000	0.584360	1.260183
6	AircraftBuoyant	data\AircraftBuoyant\ym1343.obj	.obj	178	344	344	triangles	0.025	0.025	0.025	0.710921	0.975000	0.710921	0.685921	0.950000	0.685921	1.357746
7	AircraftBuoyant	data\AircraftBuoyant\ym1344.obj	.obj	1061	1914	1914	triangles	0.025	0.025	0.025	0.674099	0.975000	0.674100	0.649099	0.950000	0.649100	1.321045
8	AircraftBuoyant	data\AircraftBuoyant\ym1345.obj	.obj	3174	6000	6000	triangles	0.025	0.025	0.025	0.601277	0.975000	0.601277	0.576277	0.950000	0.576277	1.251675
9	AircraftBuoyant	data\AircraftBuoyant\ym1346.obj	.obj	742	1374	1374	triangles	0.025	0.025	0.025	0.322079	0.354725	0.975000	0.297079	0.329725	0.950000	1.048558

Figure 12: Per-shape statistics (sample of 10 meshes)

Screenshot from the analysis output file showing vertex and face counts, face type, and bounding-box measurements for ten AircraftBuoyant shapes.

4.2.2. Step 2.2 – Statistics over the whole database

We examined the entire dataset to understand the global distribution of resolution and class membership. The histograms of vertex counts (Figure 13) and face counts (Figure 14) revealed a highly imbalanced collection, with many meshes far below or far above the level of detail needed for meaningful geometric analysis.

The scatter plot of vertices versus faces (Figure 15) showed that there was no consistent relationship between resolution and class type, indicating inconsistent modeling practices across the dataset. The class distribution (Figure 16) confirmed that some categories contain significantly more objects than others.

During the lectures, it was emphasized that most shape descriptors require meshes with comparable resolution. If one model has only 30 vertices and another has 60,000, their curvature, PCA axes, and radial features will be completely incomparable.

For this reason, the recommended working band of **5,000–10,000 vertices** was adopted. This range preserves enough geometric detail for complex models while still being computationally reasonable.

Objects outside this band, either too coarse or too dense, were resampled in the next step.

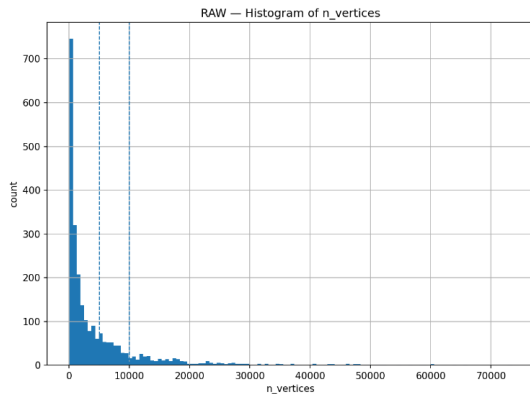


Figure 13: Histogram of vertex counts in the raw dataset, showing extreme variation from very coarse to very dense meshes.

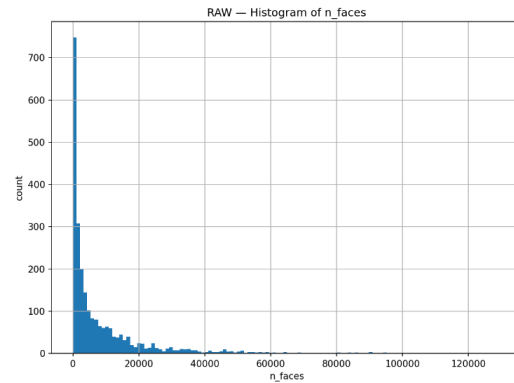


Figure 14: Histogram of face counts before resampling, illustrating large inconsistencies in mesh density.

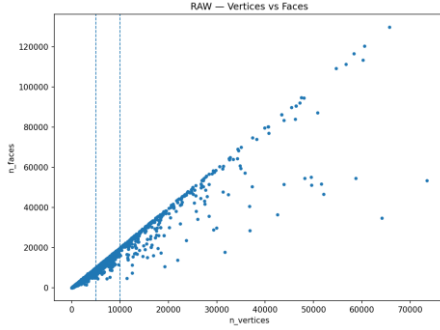


Figure 15: Scatter plot of vertices versus faces in the raw dataset. The widespread indicates inconsistent levels of detail.

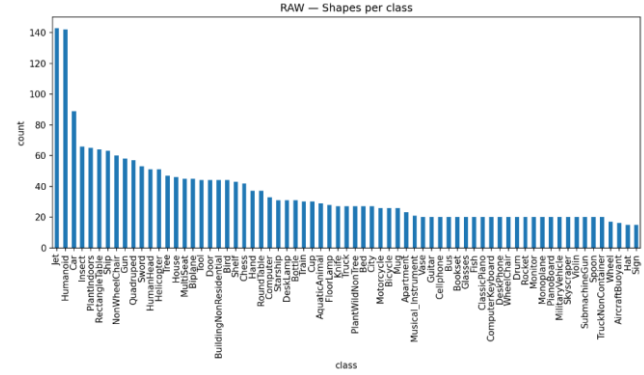


Figure 16: Number of shapes per class in the raw database.

4.2.3. Step 2.3 – Resampling outliers

To bring all meshes into the 5k–10k range, two complementary strategies were used:

Refinement (for coarse meshes)

Meshes below 5000 vertices are refined through subdivision. Subdivision increases the number of vertices while preserving the shape as much as possible. A safeguard stops the refinement process early if the mesh starts to deviate too much from its original geometry, preventing over-smoothing. This is especially important for simple meshes like Door or Tool, which start with only a handful of triangles.

Simplification for dense meshes

Meshes above 10,000 vertices are simplified using Quadric Error Metric (QEM) decimation. QEM reduces complexity while maintaining local geometry and curvature. To keep simplification stable across objects, the face-to-vertex ratio is monitored, ensuring that the algorithm does not over-collapse or distort models with thin structures (e.g., airplane wings).

This adaptive pipeline allows all shapes to converge to a consistent and workable resolution without compromising their characteristic geometry.

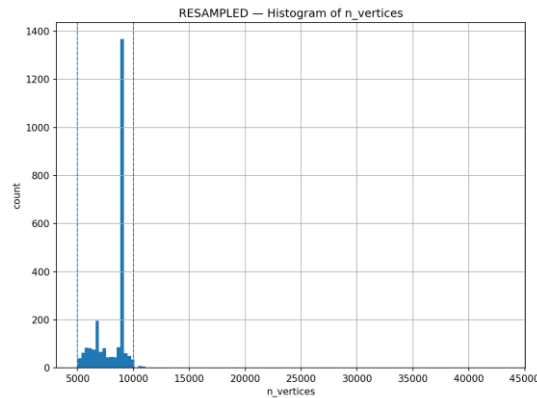


Figure 17: Vertex-count distribution after resampling. All shapes now fall inside or close to the recommended 5k–10k range.

```

== Smallest (resampled) ==
cls  n_vertices  n_faces  path
PlantIndoors  7242  4683  step2_results/resampled/PlantIndoors/m995_simplified.obj
Tree  5410  4629  step2_results/resampled/Tree/m1050_kept.obj
Tree  11349  4767  step2_results/resampled/Tree/D00612_simplified.obj
City  5135  5213  step2_results/resampled/City/D01181_kept.obj
PlantWildNonTree  7914  5274  step2_results/resampled/PlantWildNonTree/m966_simplified.obj

== Largest (resampled) ==
cls  n_vertices  n_faces  path
Vase  7872  23040  step2_results/resampled/Vase/m541_refined.obj
NonWheelChair  9986  19936  step2_results/resampled/NonWheelChair/m786_refined.obj
Bottle  9949  19872  step2_results/resampled/Bottle/D00940_refined.obj
Sword  9906  19808  step2_results/resampled/Sword/D00635_refined.obj
Humanoid  9932  19776  step2_results/resampled/Humanoid/m205_refined.obj

== Average (resampled) ==
cls  n_vertices  n_faces  path
Wheel  9000  18000  step2_results/resampled/Wheel/m745_simplified.obj

```

Figure 18: Outlier shapes after resampling. All objects now lie within acceptable resolution limits.

```

== Smallest (raw) ==
cls  n_vertices  n_faces  path
Door  16  16  data/Door/D01121.obj
Door  16  16  data/Door/m1708.obj
Door  18  17  data/Door/D01194.obj
House  25  22  data/House/m462.obj
Tool  24  25  data/Tool/m1102.obj

== Largest (raw) ==
cls  n_vertices  n_faces  path
Biplane  65698  129810  data/Biplane/m1120.obj
Chess  60557  120466  data/Chess/m1595.obj
Helicopter  58357  116645  data/Helicopter/m1388.obj
Car  60220  113420  data/Car/m1544.obj
Bicycle  56754  111446  data/Bicycle/D00462.obj

== Average (raw) ==
cls  n_vertices  n_faces  path
Door  1724  3468  data/Door/m1712.obj

```

Figure 19: Outlier shapes before resampling, showing extremely coarse and extremely dense examples.

4.2.4. Step 2.4 – Checking the resampling

To demonstrate that resampling was applied correctly, both statistical and visual checks were performed.

Statistically, resampling dramatically reduced variance: the vertex range collapsed from 16–65k in the raw dataset to roughly 100–10k after processing. The standard deviation dropped by approximately 85%, showing that resolution was effectively standardized. Visually, comparing shapes before and after resampling showed clear improvement. For example, the raw car model (Figure 20) contained large, uneven polygons that poorly represented curved surfaces.

After resampling (Figure 21), it displayed a uniform triangle layout that preserved the original proportions while matching the target resolution band.

These checks confirm that the combination of refinement and QEM simplification worked consistently across the entire dataset.

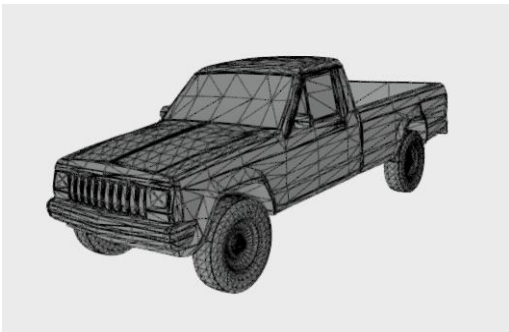


Figure 20: Car before resampling, sparse, irregular polygons cause under-representation of curved regions.

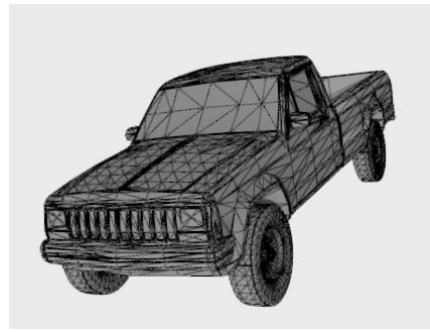


Figure 21: Car after resampling, uniform tessellation and improved surface continuity within the 5 k–10 k band (vertices 8708).

4.2.5. Step 2.5 – Normalization

After resolving resolution differences, the next step was to make all shapes comparable in position and scale. Every mesh was first translated so that its centroid lies at the origin, and then uniformly scaled so that its largest bounding-box dimension equals one. This results in each object fitting inside a unit cube, while keeping its proportions intact.

To verify normalization, two scalar checks were recorded for each mesh: the distance of the centroid from the origin, and the maximum side of the bounding box. The centroid distances (Figure 22) collapsed to values around numerical precision, indicating perfect recentering. The normalized extent values (Figure 23) formed an extremely tight cluster around 1.0, confirming uniform scaling.

Normalization does not modify the number of vertices or faces, so the distributions of resolution remain identical to the resampled stage (Figures 24 and 25). The vertices-versus-faces relation also stays unchanged (Figure 26).

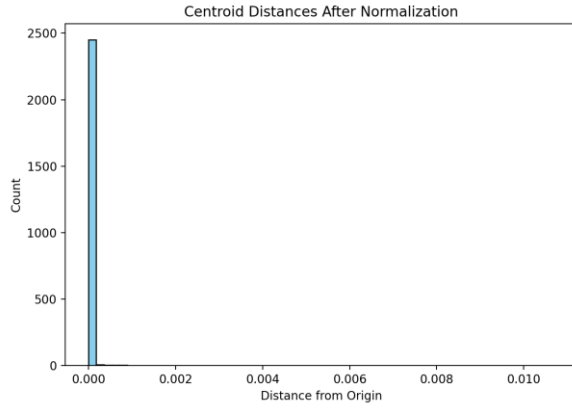


Figure 22: Centroid distances after normalization (all values near zero).

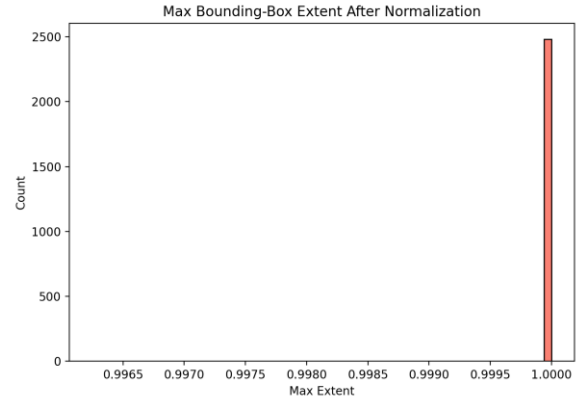


Figure 23: Normalized bounding-box extents (all values near one).

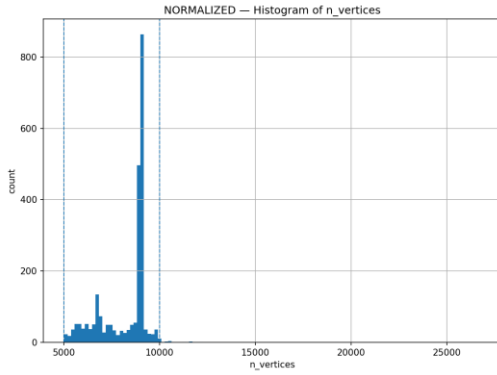


Figure 24: Histogram of vertex counts after normalization.

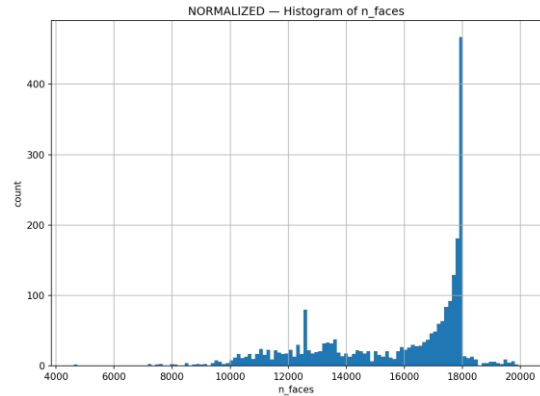


Figure 25: Histogram of face counts after normalization.

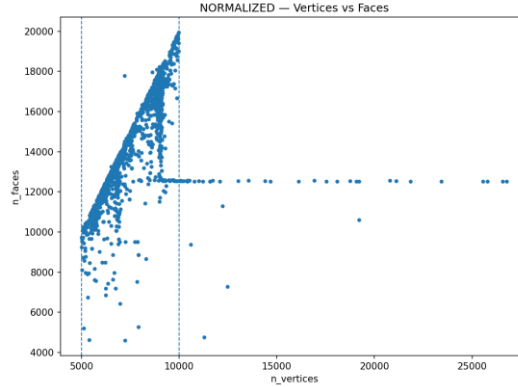


Figure 26: Vertices-versus-faces after normalization.

4.3. Results

4.3.1. Translation Verification

To verify that the translation step was applied correctly, we examined the distribution of centroid distances after normalization. Ideally, after subtracting the centroid of each mesh, the geometric center of all objects should lie exactly at the origin.

The histogram in Figure 22 clearly shows that this is the case: every centroid lies extremely close to zero, with values typically in the order of 10^{-15} . These small numbers are not actual offsets but simply floating-point rounding errors that occur when storing real numbers on a computer. No mesh shows any noticeable deviation from the origin.

This result is important because many descriptors such as PCA axes, radial signatures, and distance-based features are sensitive to where the shape is located in space. If even a few meshes remained shifted, their descriptors would be systematically biased.

The tight concentration of the centroid values around zero therefore confirms that the translation step succeeded uniformly for all shapes in the dataset.

4.3.2. Scaling Verification

A second check was performed to confirm that all meshes were scaled consistently into the unit cube. For each normalized object, we measured the maximum extent of its axis-aligned bounding box. By construction, this value should be exactly one, meaning that the mesh fits neatly inside the cube $[-1,1]^3$ after translation and scaling.

Figure 23 demonstrates that this requirement is satisfied for every single mesh. All normalized extents cluster tightly around the value 1.0, with only microscopic numerical fluctuation (again on the order of 10^{-6} to 10^{-15}). There are no outliers or irregularities.

This uniform scaling is essential for later steps. Without it, large objects (e.g., airplanes or cars) would dominate distance-based descriptors, and tiny objects (e.g., tools or doors) would appear artificially similar simply because of their small size.

Ensuring identical scale means that subsequent features measure **shape**, not **original modeling units**. The virtually identical extent values across the entire dataset confirm that this goal was fully achieved.

4.3.3. Topology and Resolution Consistency

Normalization affects only the position and scale of the mesh; it does not alter its connectivity or resolution. To confirm this, we inspected the vertex and face distributions after normalization (Figures 24 and 25).

As expected, both histograms are identical to the resampled distributions. The vertex counts remain sharply concentrated between 5,000 and 10,000 vertices, and the face counts show the same controlled density range.

Additionally, the scatter plot of vertices versus faces (Figure 26) retains the same linear relationship observed after resampling, with a stable and consistent face-to-vertex ratio across all classes. This confirms that no unintended simplification, refinement, or topological modification occurred during normalization.

Maintaining resolution consistency is critical for the validity of the system. If normalization had accidentally distorted or altered mesh density, it would undermine all the work done in the resampling stage and introduce new imbalance into the dataset.

The fact that both the distributions and the scatter relationships remain unchanged shows that the normalization stage respected the geometric structure of every mesh exactly as intended.

4.3.4. Representative Mesh Comparisons

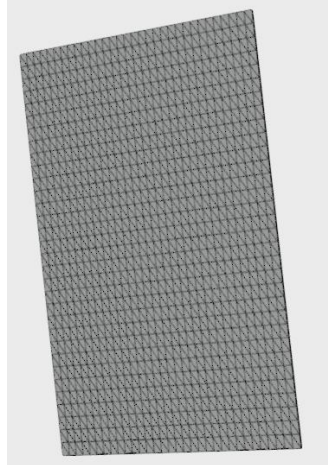
To demonstrate how the preprocessing pipeline affects meshes of different complexity, four representative objects were selected: **Door**, **Car**, **Biplane**, and **Tool**.

These shapes were chosen because they cover the full range of resolutions found in the raw dataset from extremely coarse models with only a few vertices to highly detailed meshes exceeding sixty thousand vertices. After resampling and normalization, all four objects fall within the desired resolution band and fit consistently inside the unit cube.

The vertex counts before and after normalization are summarized in Table 2, together with a short observation on each object's geometric characteristics.



*Figure 27: Raw Door (D01121)
vertices 16*



*Figure 28: Normalized Door (D01121)
vertices 8324*



Figure 29: Raw Car (m1544) vertices 60326

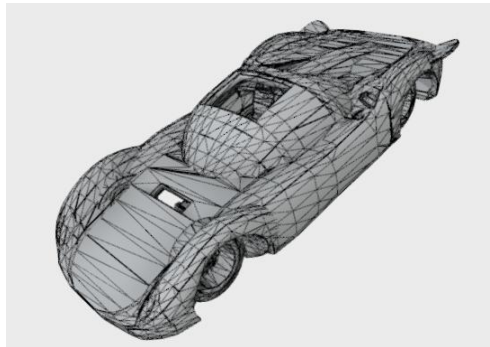


Figure 30: Normalized Car (m1544) vertices 8828



Figure 31: Raw Biplane (m1120) vertices 65722

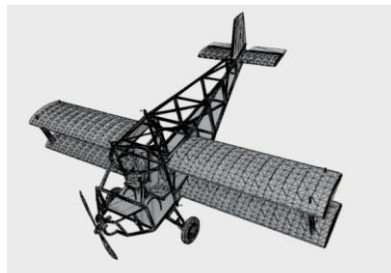
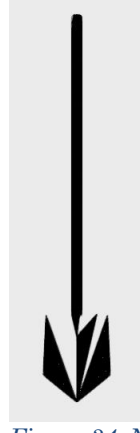


Figure 32: Normalized Biplane (m1120) vertices 8933



*Figure 33: Raw Tool (m1102)
vertices 24*



*Figure 34: Normalized Tool (m1102)
vertices 8708*

4.4. Conclusion

The preprocessing results show that the dataset has been successfully brought into a more usable and consistent state. The resampling step did what it needed to do: objects that were too coarse were refined to a reasonable level, and extremely dense meshes were simplified without losing their overall shape. The new vertex counts fall inside, or very close to, the recommended 5k–10k range. This makes the dataset much more balanced than the raw version.

Normalization also worked as expected. All meshes are now centred around the origin and scaled so that their largest dimension fits inside a unit cube. The numerical checks confirm this clearly.

This step is important because it ensures that later features and distance calculations do not reflect differences in original modelling units or object placement.

There are, however, a few practical observations worth mentioning. Some elongated objects, like the Tool, look visually stretched after scaling. This is not an error, it simply reflects the object’s true aspect ratio.

Also, even though the resampling methods preserve the overall shape, extremely coarse meshes (like those starting with ~20 vertices) will never contain the same level of detail as originally well-modelled objects. The refinement process improves them but cannot “invent” fine details that were never there.

Overall, Step 2 achieved the main goal: the dataset is now consistent in resolution, position, and scale. While some limitations are unavoidable due to the nature of the original meshes, the processed dataset is now in a much better condition for feature extraction and comparison in the next steps.

5.Step 3 - Feature Extraction

5.1 Aim

The aim of this section is to extract features from each shape in the database, after completing the normalization step.

5.2 Normalization

This section was implemented using the **Open3D 0.19.0** and **NumPy 2.3.3** libraries. The goal of normalization is to standardize the orientation, scale, and position of 3D meshes to ensure consistent feature extraction. The normalization process is divided into four sequential steps:

- a. Translation: move the barycenter to the origin

To eliminate the absolute positional differences of meshes in 3D space and make all meshes start from the same reference point. We moved the barycenter to the origin. This process was already implemented in [4.2.5](#). The below figure 36 shows the result of translation normalization. Before translation, the barycenter of the meshes were dispersed in a range far from the origin [0, 5]. After translation, all meshes were centered at the origin, with a distance of around 0.000077 (shown in the Figure37), thus demonstrating the success of translation normalization.

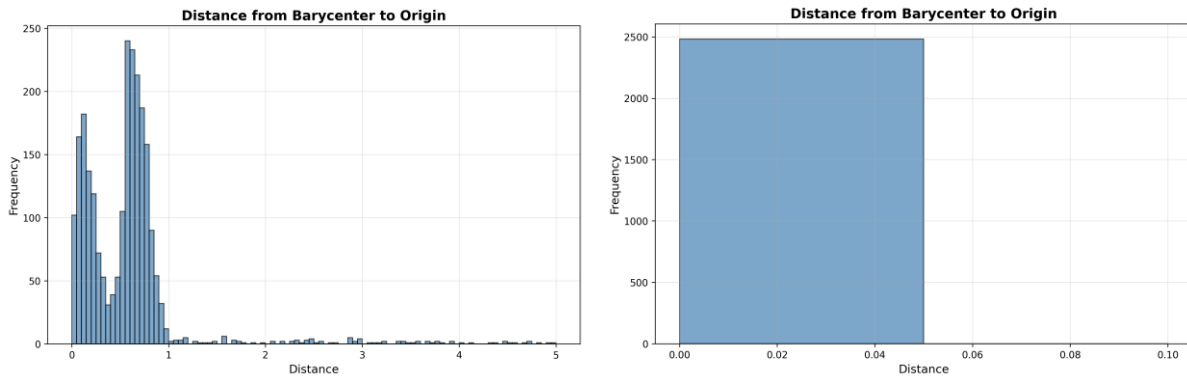


Figure 36: Distance distribution between barycenter and origin before(left) and after(right) translation

TRANSLATION (Center to Origin)		SCALE (AABB Max Extent)	
-----		-----	
Mean distance:	0.000077	AABB mean:	0.999997
Std deviation:	0.001862	AABB std:	0.000106

Figure 37: Statistics of the translation (left) and scaling (scaling) results

- b. Size normalization: scale to unit size

To eliminate scale variations and ensure all meshes have comparable sizes, we normalized each mesh to unit size by scaling it according to its Axis-Aligned Bounding Box (AABB) maximum extent. The scaling factor was computed as 1 divided by the maximum extent of the mesh's AABB, ensuring that the largest dimension of each mesh equals 1 unit after scaling. The below Figure 38 shows the result of size normalization. Before scaling (shown on the left), the AABB maximum extents of the meshes were concentrated around 1.0, with some outliers. After scaling (shown on the right), all meshes were normalized to have maximum extents very close to 1.0, with an AABB mean of 0.999997 and standard deviation of 0.000106 (shown in the Figure 37), thus demonstrating the success of size normalization.

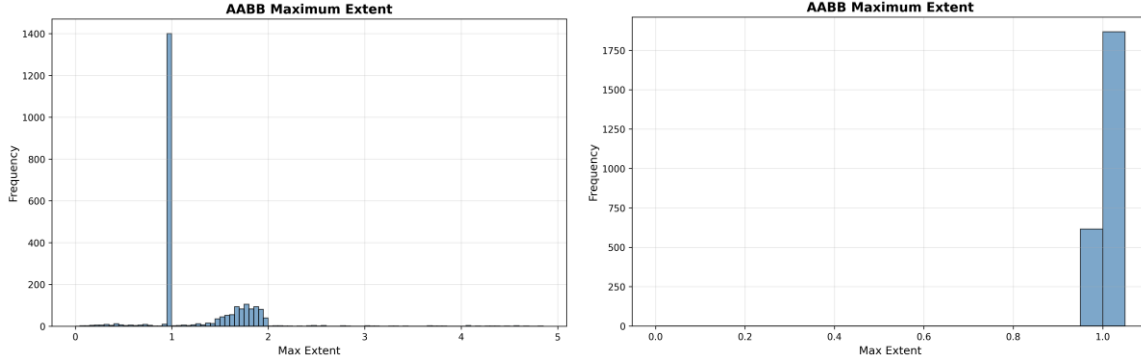


Figure 38: AABB maximum extent distribution before (left) and after (right) size normalization

c. Pose alignment: align to principal axes

To eliminate rotational variance among meshes, the coordinate frame of each shape was realigned to its principal axes. The process began by computing the covariance matrix of the vertex coordinates. This was implemented using NumPy functions `np.cov()`. The covariance matrix captures how the coordinates of the mesh vary along each spatial axis. Its eigenvectors and eigenvalues describe the principal directions and their relative importance. Function `np.linalg.eig()` was used to compute both the eigenvalues and eigenvectors, and function `np.argsort()` was used for finding the major and medium eigenvectors. Then each vertex p_i of the mesh is then projected onto these principal axes to yield the aligned coordinates:

$$\begin{aligned} x_i^{updated} &= p_i \cdot e_1 \\ y_i^{updated} &= p_i \cdot e_2 \\ z_i^{updated} &= p_i \cdot (e_1 \times e_2) \end{aligned}$$

Where e_1 and e_2 are the major and medium eigenvectors respectively. Finally, mesh's x-axis was aligned with the direction of maximum spatial variance, and its y-axis was aligned with the orthogonal direction of secondary variance, and its z-axis which is computed via cross product ensured a consistent right-handed orientation.

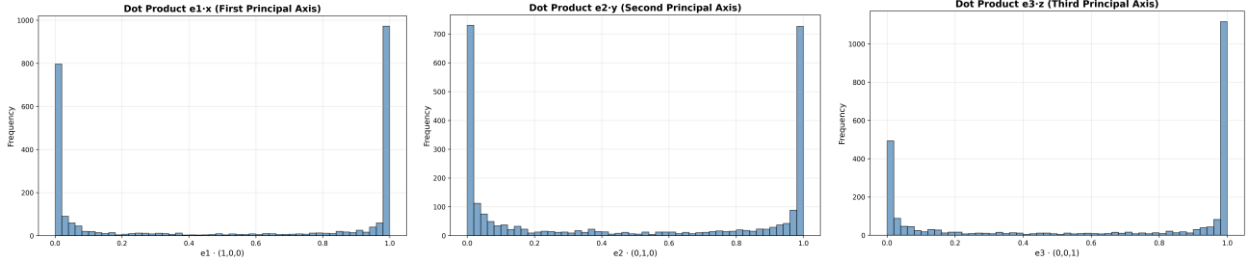


Figure 39: Distribution of dot products between principal axes and coordinate axes before pose alignment

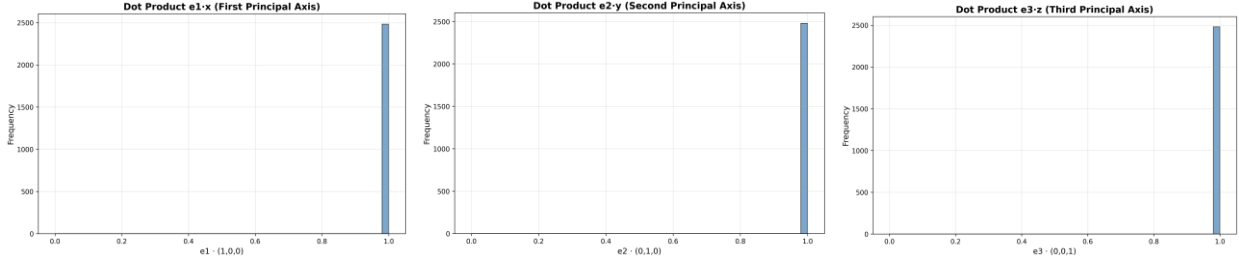


Figure 40: Distribution of dot products between principal axes and coordinate axes after pose alignment

Figures 39 and 40 illustrate the success of the pose alignment. Before alignment (shown in Figure 39), the dot products between the meshes' principal axes and the coordinate axes exhibited a bimodal distribution spanning the range $[0, 1]$. All principal axes showed significant dispersion, with peaks located at 0 and 1 respectively. After pose alignment (shown in Figure 40), we observed a significant improvement. All three distributions converged to close peaks close to 1, and the dot products of all meshes were very close to 1.

d. Flipping test: ensure consistent orientation

Although eigen decomposition can align the mesh with its principal axes, but the direction of each eigenvector is arbitrary. Therefore, a flipping test was performed to make sure that the side of the axis that contains the most mass of the shape is in the positive half of the axis. This is done by computing a moment-like measure f_i along each axis using this formula:

$$f_i = \sum_t \text{sign}(C_{t,i})(C_{t,i})^2$$

Where t is the number of triangles in the mesh and $C_{t,i}$ is the i^{th} coordinate of the center of triangle t . The sign formula returns 1 if the input is positive and -1 if it is negative. Then each coordinate is updated using this formula:

$$\begin{aligned} x_i^{\text{updated}} &= x_i \text{sign}(f_0) \\ y_i^{\text{updated}} &= y_i \text{sign}(f_1) \\ z_i^{\text{updated}} &= z_i \text{sign}(f_2) \end{aligned}$$

This guarantees that all meshes share a consistent orientation, with their major mass aligned toward the positive side of each axis.

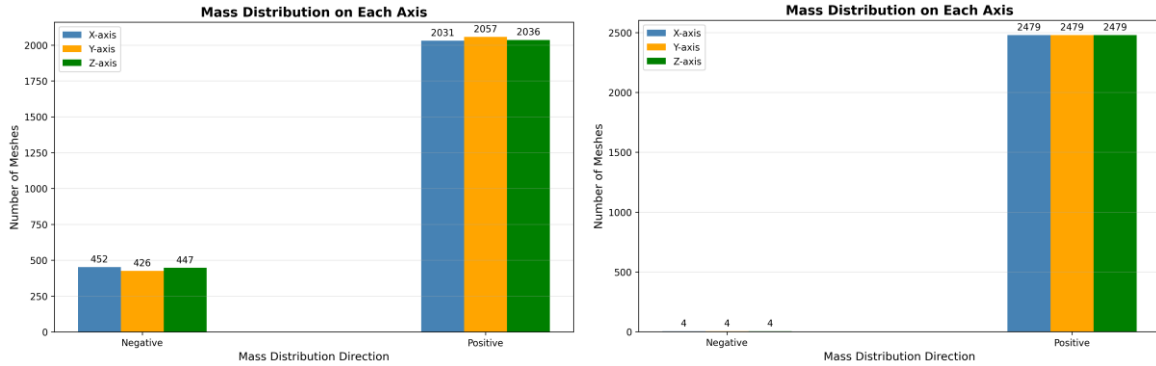


Figure 41: Mass distribution on each principal axis before (left) and after (right) flipping test

Figure 41 shows the success of the flipping test. Before applying the test, the mass distribution of the meshes was not consistent across the X, Y, and Z axes. A portion of the mass lay on the negative axes. For example, on the X-axis, 452 triangle centroids were located in the negative direction and more than 2031 in the positive direction. After performing the flipping test, all three axes exhibited mass in the positive direction. The mass distribution along the X, Y, and Z axes became almost positive, while the negative sides were reduced to near zero.

5.3 3D shape descriptors

This section was implemented using the **Open3D** 0.19.0, **SciPy** 1.16.2 and **NumPy** 2.3.3 libraries.

After normalization, we extracted the features of the 3D shape so that each 3D shape was converted into a comparable feature vector of fixed length. Six 3D elementary descriptors was calculated first.

a. Surface area

Function `mesh.get_surface_area()` was used to get the surface area of the mesh. It traverses all triangles, calculates the two edge vectors for each triangle and calculates the cross product, and finally divides by 2 to get the area of each triangle. After accumulating the areas of all triangles, the final surface area is obtained.

For a single triangular:

$$A_{\Delta} = \frac{1}{2} \| (v_2 - v_1) \times (v_3 - v_1) \| \quad (v_i \text{ is the } i\text{-th vertex (vector)})$$

For total surface area:

$$S = \sum_{i=1}^m A_{\Delta_i}$$

b. Compactness (with respect to a sphere)

We used the following equation to calculate the compactness, which measures how similar the shape is to a sphere.

$$C = \frac{S^3}{36\pi V^2}$$

Where S represents surface area and V represents volume.

The volume of the mesh is calculated by the Tetrahedron Decomposition Method. It connects each triangle of the mesh to the origin to form a tetrahedron, calculates the signed volume of each tetrahedron, and then adds up all the signed volumes and takes the absolute value, because the volumes facing outward and inward cancel each other, leaving only the net volume inside the mesh.

$$V = \frac{1}{6} |\sum_i (v_1 \times v_2) \cdot v_3|$$

where

$$v_1 = x_1 - o$$

$$v_2 = x_2 - o$$

$$v_3 = x_3 - o$$

(x_i is the vertex of the triangle, o is the origin)

c. 3D rectangularity

We used the following equation to calculate the 3D rectangularity, which is used to measure the geometric regularity of a shape, that is, how much the shape resembles a cuboid.

$$R = \frac{V}{V_{OoB}}$$

(V is the calculated volume, V_{OoB} is the volume of Oriented Bounding Box)

d. Diameter

Diameter is the largest distance between any two surface points. Because the convex hull is the smallest convex polyhedron that encloses all vertices of the mesh, so we calculated the maximum distance between any two vertices on the convex hull boundary as the diameter. So we computed the convex hull of the vertex set firstly and then extracted the vertices lying on the convex hull boundary and used function `pdist(hull_points)` to calculate all pairwise Euclidean distances between these vertices. Finally, we used the function `np.max()` to select the maximum distance.

e. Convexity

Convexity quantifies the degree to which a shape is convex by comparing its volume to that of its convex hull:

$$C = \frac{V}{V_{hull}}$$

Where V is the shape volume, V_{hull} is the convex hull volume.

f. Eccentricity

Eccentricity is the ratio of the largest to the smallest eigenvalues of the covariance matrix, which can be used to determine whether the shape is round or elongated.

$$E = \frac{|\lambda_1|}{|\lambda_3|}$$

Where λ_1 is the largest eigenvalue and λ_3 is the smallest eigenvalue of the covariance matrix.

Since we calculated simple global descriptors, we also calculated shape property descriptors which are distributions. Because the shape property distribution is very sensitive to the good distribution of sampling points on the shape surface. Therefore, we applied weighted sampling based on area to implement the uniform sampling when calculating the distribution descriptors. Open3D's built-in `uniform_sample_mesh()` function randomly selects a specified number of sampling points from the entire surface based on the area ratio of the mesh triangles, thus avoiding the distribution bias caused by sampling only from vertices.

a. A3: angle between 3 random vertices

To implement the A3 descriptor, we used a nested loop sampling method. First, we calculated the number of iterations k per loop based on the target sample number n ($k^3 \approx n$). Then, three different vertices were randomly selected through three nested loops. For each set of three vertices A, B, and C, vertex A was used as the corner point, and the angle between vectors $\vec{AB}$ and $\vec{AC}$ was calculated. The calculation was performed by first taking the dot product of the two vectors and dividing it by the product of their magnitude to obtain the cosine value. The inverse cosine function was then used to get the radian angle, which was then converted to degrees.

b. D1: distance between barycenter and random vertex

To implement the D1 descriptor, we first computed the barycenter of the mesh by taking the mean of all vertices. Then, we randomly sampled n vertices and calculated the Euclidean distance from each sampled vertex to the barycenter using the L2 norm. These distances were collected into an array, representing the distribution of random vertex positions relative to the mesh center.

c. D2: distance between 2 random vertices

To implement the D2 descriptor, we used a nested loop sampling method. First, we calculated the number of iterations k per loop based on the target sample number n ($k^2 \approx n$). Then, two different vertices were randomly selected through two nested loops, ensuring the two selected vertices were distinct. For each pair of vertices, we calculated the Euclidean distance between

them using the L2 norm. These distances were collected into an array, representing the distribution of pairwise distances within the mesh.

d. D3: square root of the area of the triangle given by 3 random vertices

To implement the D3 descriptor, we used a nested loop sampling method. First, we calculated the number of iterations k per loop based on the target sample number n ($k^3 \approx n$). Then, three different vertices were randomly selected through three nested loops to form triangles. For each set of three vertices A , B , and C , we calculated the triangle area using the cross product of vectors $\vec{AB}$ and $\vec{AC}$, where the area equals half the magnitude of the cross product. Finally, we took the square root of each area value. These square root values were collected into an array, representing the distribution of triangle sizes within the mesh.

e. D4: cube root of the volume of the tetrahedron formed by 4 random vertices

To implement the D4 descriptor, we used a nested loop sampling method. First, we calculated the number of iterations k per loop based on the target sample number n ($k^4 \approx n$). Then, four different vertices were randomly selected through four nested loops to form tetrahedra. For each set of four vertices A , B , C , and D , we calculated the tetrahedron volume using the scalar triple product formula:

$$V = \frac{\left| \left((A - D) \cdot ((B - D) \times (C - D)) \right) \right|}{6}$$

Finally, we took the cube root of each volume value. These cube root values were collected into an array, representing the distribution of tetrahedron sizes within the mesh.

5.4 Results

The results of the 3D elementary descriptor :



Figure 42: A human head named D00610.obj in HumanHead class

HumanHead 000610_simplified.obj 5.387698491994287 0.8619356664622685 1.8612548336969121 0.33020688109230234 1.608954790290326 0.7660172669039317 1.2923628210069407

Figure 43: The statistics 3D elementary descriptors for D00610.obj in HumanHead class

Figures 42 and 44 show meshes of a human head and two similar fish. In Figures 43 and 45, the statistics of 3D elementary descriptors are shown. As for volume, both are

less than 1, which is consistent with the rule that the volume of the shape due to size normalization should be in the range of [0,1]. As for compactness, the human head is relatively low, close to 1, indicating it is more spherical than the fish. As for rectangularity and convexity, it's reasonable for the human head to be larger than the fish's. Firstly, the human head is relatively full, almost spherical, and its shape can be well contained within a compact bounding box, thus making it nearly rectangular. On the other hand, the fish is slender and pointed, and most of its area is empty, especially the head and tail. Therefore, the fish occupies a smaller proportion of its bounding box, resulting in less rectangularity. Regarding convexity, while the human head has details (ears, nose, etc.), its overall outline is relatively smooth, with little difference from its convex hull. However, the fish has numerous protrusions and depressions, such as the dorsal, pelvic, and caudal fins. These structures create a significant difference between the actual volume and its convex hull, thus significantly reducing convexity. As for eccentricity, the human head's value approximates a sphere, and the fish's value also matches its slender, streamlined shape.

For two similar objects in the same class, their results for 3D elementary descriptors are very close. In figure 44, m55.obj (left) has 9032 vertices, 17078 faces and m56.obj (right) has 5814 vertices, 10893 faces. These two objects are extremely similar not only for their shapes, but also on their 3D elementary descriptors values, figure 45 shows the similarity.

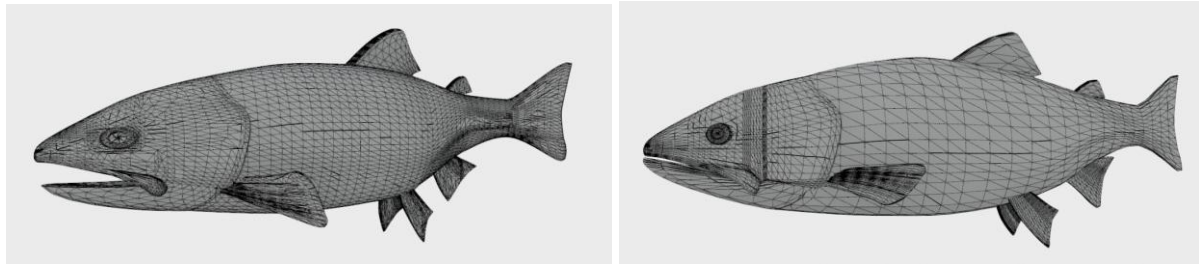


Figure 44: 2 similar objects in aquatic animal class (left: m55.obj; right: m56.obj)

class_name	file_name	surface_area	volume	compactness	rectangularity	diameter	convexity	eccentricity
AquaticAnimal	m53_full_norm.obj	0.4136116165601731	0.0040376118536782245	17.163088045508054	0.15771691271791433	0.9580436045096148	0.369640038517928	70.12881145623851
AquaticAnimal	m54_full_norm.obj	0.5280588805644733	0.010446345826348993	11.885145907607942	0.1689489698408279	0.9754414958917265	0.5352981794776155	41.39062252520367
AquaticAnimal	m55_full_norm.obj	0.4612857370869394	0.007778242962660906	14.344806571285417	0.14611590836880375	0.9630326585771222	0.38058647112231664	54.477020359533434
AquaticAnimal	m56_full_norm.obj	0.4413978461840741	0.007067269980616767	15.224214380652436	0.14905597554196423	0.9630192589623965	0.5071356714782239	58.30792432929203

Figure 45: The statistics 3D elementary descriptors for m55.obj and m56.obj in aquatic animal class

class_name ...	file_name (2)	surface_area (3)	volume (4)	compactness (5)	rectangularity (6)	diameter (7)	convexity (8)	eccentricity (9)
class_name	file_name	surface_area	volume	compactness	rectangularity	diameter	convexity	eccentricity
Apartment	000045_full_norm.obj	8.147824532283282	0.03010092463853131	5278.529614452067	0.013887367444887088	1.8917717110666066	0.04223163668248263	3.4321323092147797
Apartment	000086_full_norm.obj	4.851703944526922	0.4547781969586773	4.63368694060507	0.31794818160193885	1.0251355543274417	0.0148864377351667	12.676716433382513
Apartment	000138_full_norm.obj	12.11090985105351	0.14246421612197832	773.8670493992036	0.12218689162580895	1.9580629429892717	0.165526946667764	6.256249308815516
Apartment	000156_full_norm.obj	39.767403489918514	0.22535458759365794	10949.56422121247925	0.09559470308985557	1.9682368559781012	0.13942633339593188	7.640698420525227
Apartment	000310_full_norm.obj	18.243659450507347	0.12939828298219833	3206.444853807626	0.11730155847221105	1.953019748046808	0.152839308027074	10.636297963990865
Apartment	000348_full_norm.obj	5.1529179172551824	0.5310721584644673	4.249335089215377	0.3461634957775126	1.7509557397217868	0.6525821717157714	40.62716397696724
Apartment	000366_full_norm.obj	8.066724499895718	0.8680151794823881	6.148716947788293	0.5250709781509697	1.811268758608127	0.7512518307809303	3.773906192883198
Apartment	000478_full_norm.obj	10.609380108265048	0.41373220442415726	61.48491922772452	0.1830178753383927	1.8612326534321781	0.43840043950532226	4.988522968426734
Apartment	000496_full_norm.obj	5.548856752286743	0.6637788528056759	3.428556748339232	0.5340273585549477	1.7647984217263084	0.8537485058657645	2.640441179902078
Apartment	000626_full_norm.obj	11.340068897262885	0.412851294688041	75.64973374403495	0.35284825521110156	1.8728963991762257	0.546156334549521	6.286200020088688
Apartment	000639_full_norm.obj	13.963121639377711	0.570200893800599	74.03539473018643	0.3138451609280176	1.7856051517109095	0.6115834123922949	3.0447666060233973
Apartment	000651_full_norm.obj	39.77765238211435	0.2254586427480514	10947.91191397612	0.08759169264265727	1.9602364938763053	0.13949077157283976	7.74086748315646
Apartment	000681_full_norm.obj	28.203367716263916	0.6884284968476419	418.5360390756931	0.31865726762350977	1.9242921638918828	0.6157013994066025	8.425292446579018
Apartment	000780_full_norm.obj	5.268883465877784	0.28904930574451226	15.479593257485465	0.4193453994701369	1.8954834344431493	0.46031311153079923	23.874755880521413
Apartment	000792_full_norm.obj	4.4127724121054035	0.9915962001318455	2.3714577975749243	0.3635857828836882	1.7871391773669687	0.9147746833502634	5.071002379233981
Apartment	000869_full_norm.obj	7.264454528175496	0.48937571468357615	14.1655649190895637	0.5062142731877796	1.8363719979241002	0.8276033985669067	12.623254109337465
Apartment	000868_full_norm.obj	10.60901233975414	0.4892461272856885	63.0395181003877	0.2371020720461873	1.8612317050513432	0.4336421544736707	5.011089212746879
Apartment	000849_full_norm.obj	7.388655868379053	0.7293571248923162	6.704455079712976	0.5425214639090371	1.8792448590384192	0.6428191263379345	3.4589424641952586

Feature Statistics:

- Total columns: 329
- Metadata: 2 (class_name, file_name)
- Single-value features: 7
- Distribution features: 320 (5 × 64 bins)

Single-Value Feature Summary:

	surface_area	volume	compactness	rectangularity	diameter	convexity	eccentricity
mean	12.027230	0.397425	1770.931743	0.308398	1.731711	0.543600	10.402205
std	10.258727	0.285414	3474.652856	0.190461	0.309407	0.312853	11.272320
min	2.988205	0.030101	2.371458	0.013887	0.955433	0.042232	1.706580
max	39.777652	0.991596	10949.564221	0.791092	1.960237	1.250131	43.540046

Figure 46: The statistics 3D elementary descriptors for apartment class

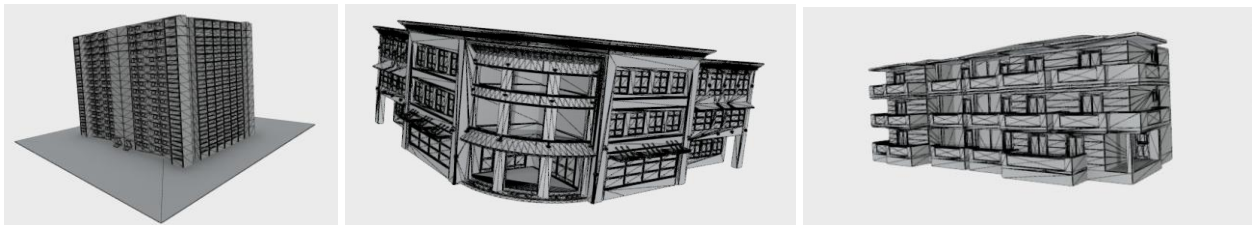


Figure 47: 3 objects in apartment class

For the apartment class, the surface area can vary greatly. The complexity of the object's shape varies greatly. In figure 47, the left apartment is highly complex, comparing with the right one, which is more simple. The volume is relatively uniform. Because the calculation formula for Compactness is based on surface area and volume, it can vary greatly and have outliers. Diameter is relatively stable, with little fluctuation within the same class. Convexity and Eccentricity also vary significantly. Figure 46 shows the great varying on 3D elementary descriptors values.

class_name (1)	file_name (2)	surface_area (3)	volume (4)	compactness (5)	rectangularity (6)	diameter (7)	convexity (8)	eccentricity (9)
class_name	file_name	surface_area	volume	compactness	rectangularity	diameter	convexity	eccentricity
AquaticAnimal	m51_full_norm.obj	0.41361516165401731	0.0046376118536782245	17.16308045508054	0.15771691271791433	0.9580436045096148	0.369646038517928	70.12881145623851
AquaticAnimal	m54_full_norm.obj	0.5280588805644733	0.010466345826348903	11.085145907607942	0.1689489698408279	0.9764414958917265	0.5352981794776155	41.39062252520367
AquaticAnimal	m55_full_norm.obj	0.4612857370869394	0.007778242942640906	14.344806571285417	0.14611590836880375	0.9630326585771222	0.38058447112231664	54.477020359533434
AquaticAnimal	m56_full_norm.obj	0.4413978461840741	0.00706726990816767	15.224214380652436	0.14905597554196423	0.9630192589623965	0.5071356714782239	58.30792432929203
AquaticAnimal	m57_full_norm.obj	0.4519359395766751	0.006482104392202089	18.278971355383644	0.16103564481716073	0.9607914622023048	0.490564770981525	81.05965749347052
AquaticAnimal	m58_full_norm.obj	0.798032396427725	0.04502373169802834	2.2147984913616556	0.2024976735665409	0.9457097566523096	0.685708477461819	5.062935868960158
AquaticAnimal	m59_full_norm.obj	0.2485425536641483	0.005648929549803376	4.264194785101139	0.30156670485906206	0.853555012016724	0.8678313957423697	65.92285541520106
AquaticAnimal	m60_full_norm.obj	0.5412001126164647	0.010701458331481549	11.664995760000012	0.09733175854471883	0.952415632177609	0.32317800065713553	31.24452694026527
AquaticAnimal	m61_full_norm.obj	0.577777839002083	0.009510878181903364	5.2780492236154	0.1645965010588065	0.9386497424692137	0.4226151350986592	43.95154383453428
AquaticAnimal	m62_full_norm.obj	0.5478616978516309	0.010421243531380513	13.388174123806706	0.28042038114854532	0.9373686019376447	0.5357611190020596	150.98260456364522
AquaticAnimal	m63_full_norm.obj	1.291010183894778	0.04504256568381220	4.091992541293935	0.1377591309928495	0.9787815229738874	0.5219344531228426	3.5569614060454366
AquaticAnimal	m64_full_norm.obj	0.4261028812957029	0.00867432279483680	9.103979511434071	0.12349531857089006	0.9365933074763623	0.44459583149450815	39.556091651173304
AquaticAnimal	m65_full_norm.obj	0.5928430488634988	0.008902188890553206	23.247339422067366	0.049485621036499721	0.9238191871435739	0.20939486371907862	15.720667651793214

Feature Statistics:

- Total columns: 329
- Metadata: 2 (class_name, file_name)
- Single-value features: 7
- Distribution features: 320 (5 × 64 bins)

Single-Value Feature Summary:

	surface_area	volume	compactness	rectangularity	diameter	convexity	eccentricity
mean	0.503755	0.013477	9.371141	0.134602	0.950261	0.440286	41.063428
std	0.252960	0.013452	5.363649	0.057575	0.029381	0.147208	28.957889
min	0.248543	0.004948	2.216798	0.049486	0.853556	0.209395	3.556961
max	1.353660	0.065043	23.247339	0.301567	1.043046	0.867831	150.982605

Figure 48: The statistics 3D elementary descriptors for aquatic animal class

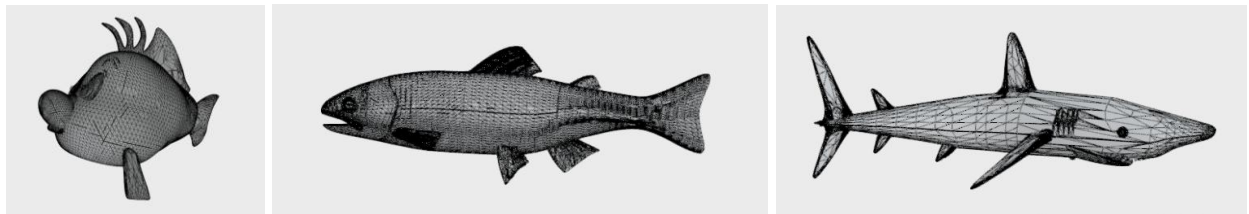


Figure 49: 3 objects in aquatic animal class

For the aquatic animal class, except for rectangularity and diameter, which show relatively stable values. The other 3D elementary descriptors, surface area, compactness, convexity, and eccentricity, all show large variations. Figure 48 shows the statistics of 3D elementary descriptors for aquatic animal class. This indicates that although these three objects all belong to the aquatic animal class, their geometric characteristics vary greatly. Figure 49 also reflects the fact that the geometric features vary greatly within the same class.

Therefore, even within the same class, geometric features can vary significantly. This means that simple threshold-based classification based on 3D elementary descriptors is not feasible. Instead, we need to use distribution-based shape descriptors (A3, D1, D2, D3, D4) to capture the underlying shape patterns that distinguish different object classes.

The results of the shape property distribution descriptors:

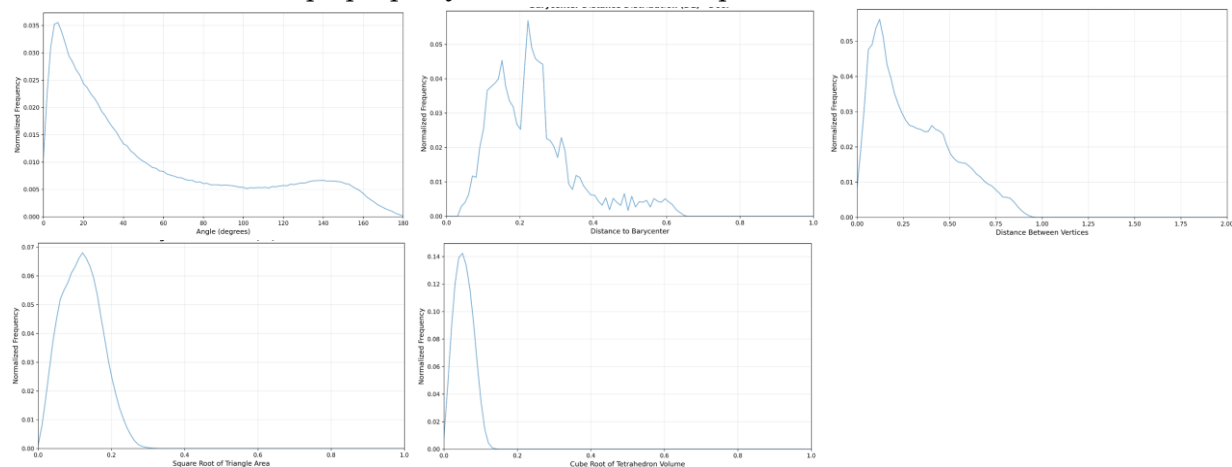


Figure 50: Normalized histogram for A3, D1, D2, D3, D4 of m55.obj in aquatic animal class (randomly sampling of vertex indices)

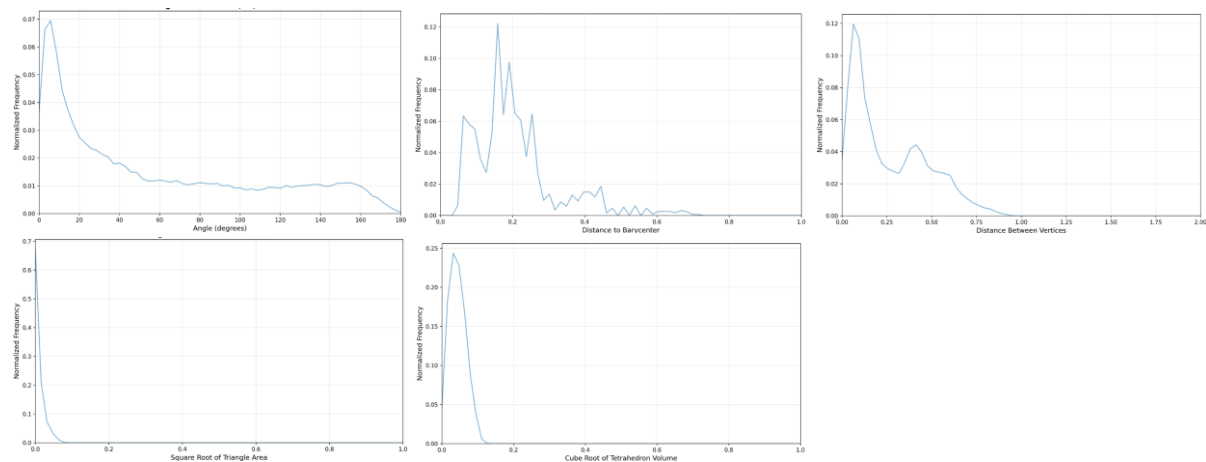


Figure 51: Normalized histogram for A3, D1, D2, D3, D4 of m56.obj in aquatic animal class (randomly sampling of vertex indices)

For two similar objects m55.obj and m56.obj in the AquaticAnimal class (shown in Figure 44), we calculated the distribution descriptor by uniform sampling of vertex indices, which results in a large difference in the distribution descriptors for two similar objects (shown in Figure 50, 51). This is because of that the shape property distribution is very sensitive to the good distribution of sampling points on the shape surface, and vertex-index sampling does not guarantee uniform distribution on the surface, especially when the mesh has varying triangle densities. Therefore, we try to use weighted sampling based on triangle area to calculate the distribution descriptor. Figure 52 and Figure 53 show that computing the distribution descriptor using a weighted sampling method based on triangle area yields two very similar results.

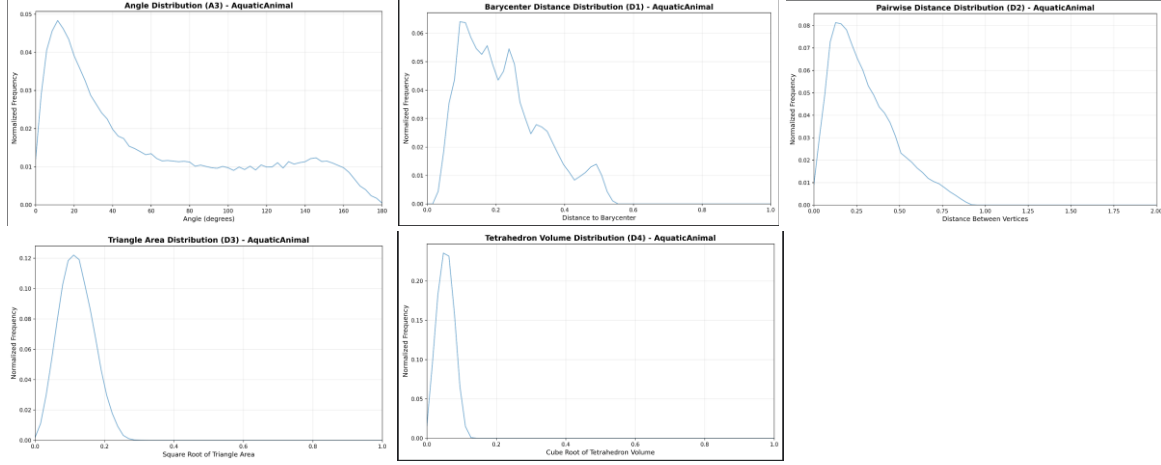


Figure 52: Normalized histogram for A3, D1, D2, D3, D4 of m55.obj in aquatic animal class (weighted sampling based on area)

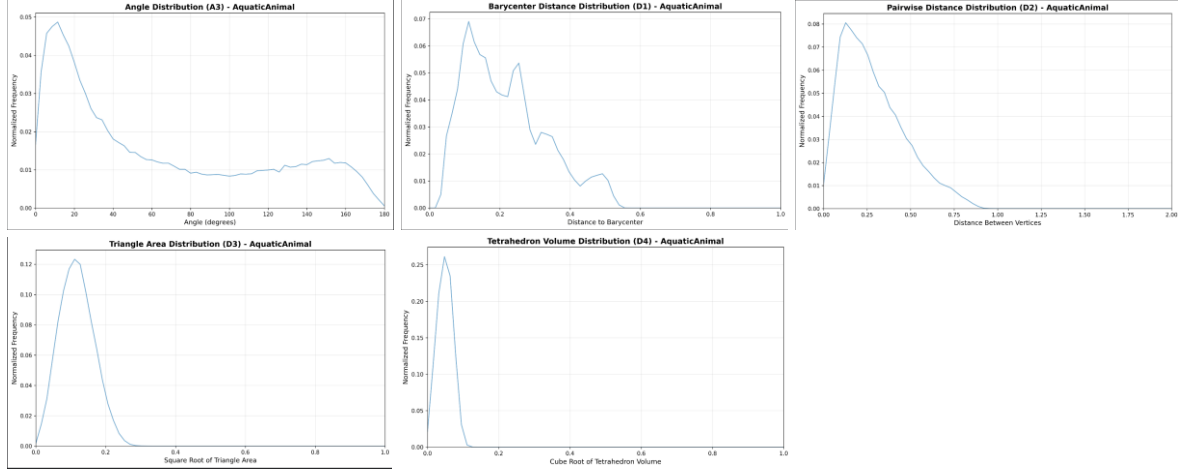


Figure 53: Normalized histogram for A3, D1, D2, D3, D4 of m56.obj in aquatic animal class (weighted sampling based on area)

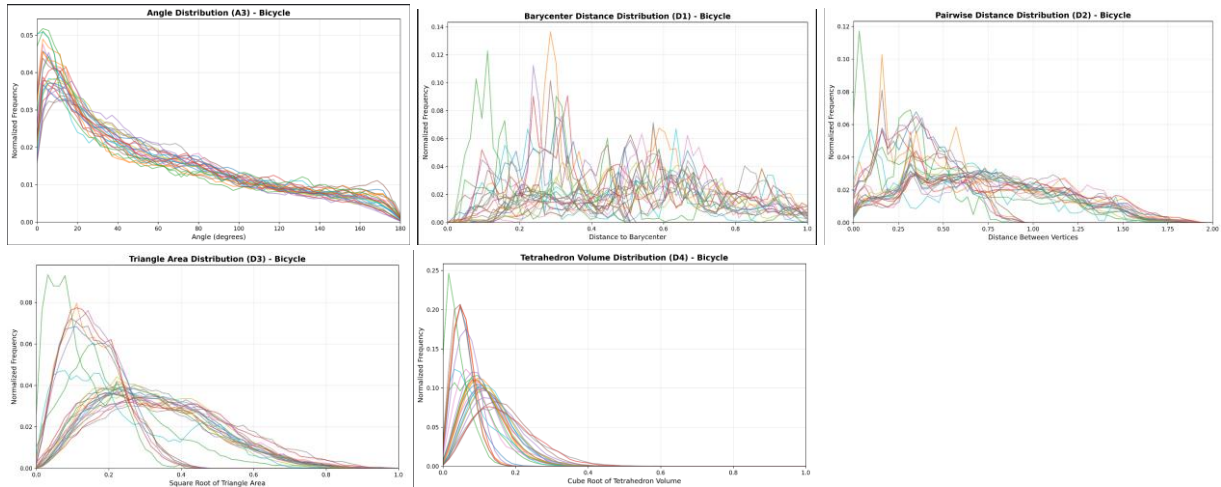


Figure 54: Normalized histogram for A3, D1, D2, D3, D4 of the bicycle class (randomly sampling of vertex indices)

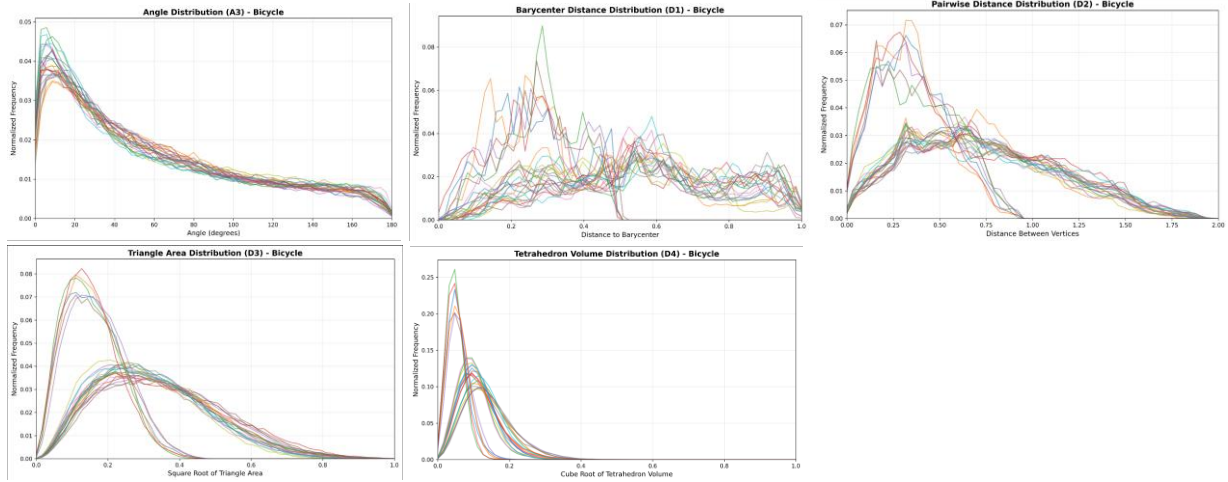


Figure 55: Normalized histogram for A3, D1, D2, D3, D4 of the bicycle class (weighted sampling based on area)

Figure 54 shows the distribution descriptors for the bicycle class calculated using randomly sampling of vertex indices. Figure 55 shows the distribution descriptors for the bicycle class calculated using weighted sampling based on triangle area. For A3, Figure 54 exhibits scattered curves with noticeable noise, while Figure 55 shows more concentrated and smoother curves. For D1, Figure 54 shows multiple, inconsistent peaks, while Figure 55 shows concentrated, similar peaks. For D2, the curves in Figure 54 overlap poorly, while those in Figure 55 overlap highly. For D3 and D4, the peak positions in Figure 54 are scattered, while those in Figure 55 are well aligned. Therefore, Figure 55 (area-weighted sampling) demonstrates better stability and consistency across all descriptors, with smoother curves and more accurate peak alignment.

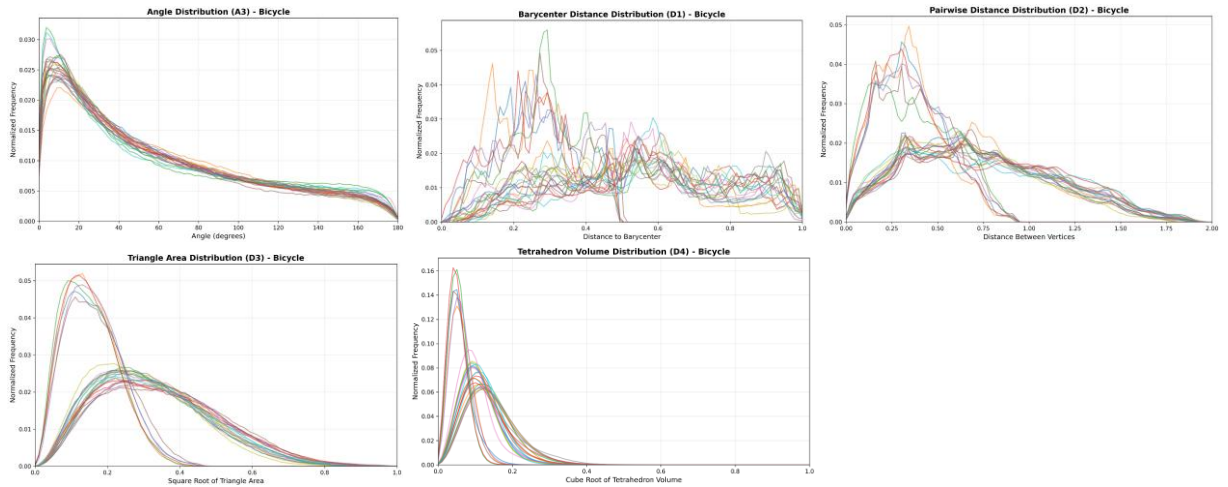


Figure 56: Distribution descriptors of the bicycle class (uses 150,000 combinations and 64 histogram bins)

As shown in Figure 55 and Figure 56, the normalized histograms of the A3, D1, D2, D3, and D4 descriptors for the bicycle class were computed using two different parameter settings.

In Figure 55, we used 1,000,000 combinations and 100 histogram bins, while Figure 56 shows the results obtained with 150,000 combinations and 64 histogram bins. The results obtained with the two different parameters setting are very similar. When using 1,000,000 combinations, it takes approximately 47 seconds to process a mesh, while using 150,000 combinations only takes approximately 8 seconds. Therefore, for efficiency reasons, we used 150,000 combinations and 64 histogram bins to calculate the Normalized histogram for A3, D1, D2, D3, and D4 of all classes.

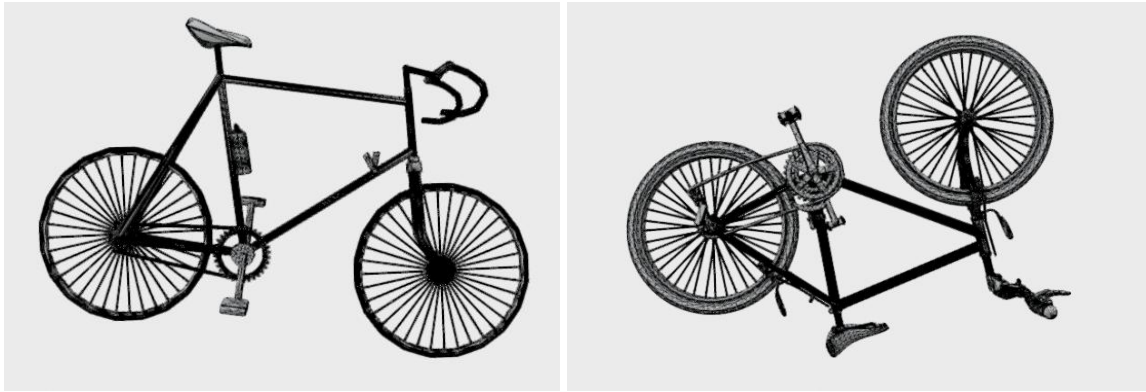


Figure 57: Two different orientation of the bicycle meshes

In Figure 54-56, we found that within the same type of bicycle meshes, the D2, D3, and D4 descriptors exhibit two different waveforms, primarily due to differences in the meshes' spatial orientation. Figure 57 shows two different orientation of the bicycle meshes. These descriptors reflect the distribution of point-to-point distances, the area distribution of triangles formed by three points, and the volume distribution of tetrahedrons formed by four points, respectively, and their values are entirely determined by the spatial geometric relationships between the point clouds. When the bicycle is placed in a normal upright position, the geometry of the two wheels and the frame exhibits a stable and regular distribution in space; however, when the bicycle is inverted or placed in other postures, the relative spatial positions of the wheels, frame, and other components change, causing changes in the distance relationships between points and the local geometry. Therefore, even for the same bicycle, significant differences in the D2, D3, and D4 distributions can occur simply due to different orientations, resulting in two different waveform patterns. Although we have previously performed normalization and verified its success using histograms, some meshes still exhibit inconsistent orientations. We thought that it is due to the limitation of the moment test. In theory, this approach allows the algorithm to infer the principal axes and distinguish their positive and negative directions based on triangle density. However, when different regions of a mesh contain uneven geometric detail, this method may produce inconsistent results. Therefore, the generation of two waveforms of the same type is likely due to differences in mesh quality and limitations of the moment test.

The figures 58-62 show the A3, D1, D2, D3, and D4 shape property descriptors for different classes. For these descriptors, the curves within each figure are relatively similar, and the overall curves vary between figures (this does not apply to all classes, as some plots are very similar). In each figure, the curves are well grouped, indicating that shapes from the same class produce similar histograms. However, within these plots, there are a few anomalous curves, likely due to the quality of individual meshes within that class.

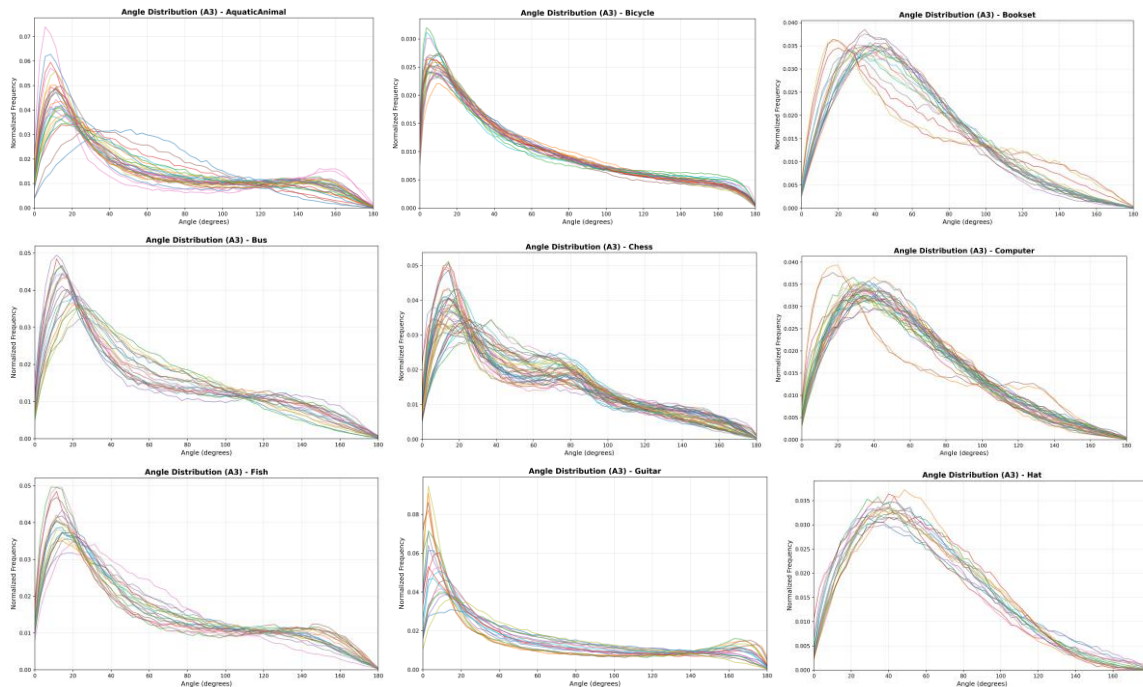
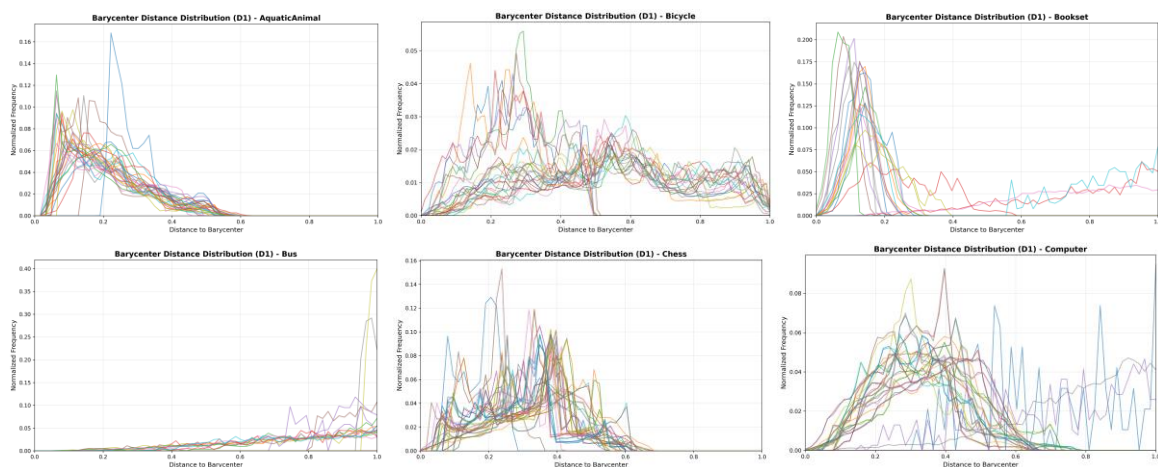


Figure 58: A3 descriptors for all shapes, grouped per class



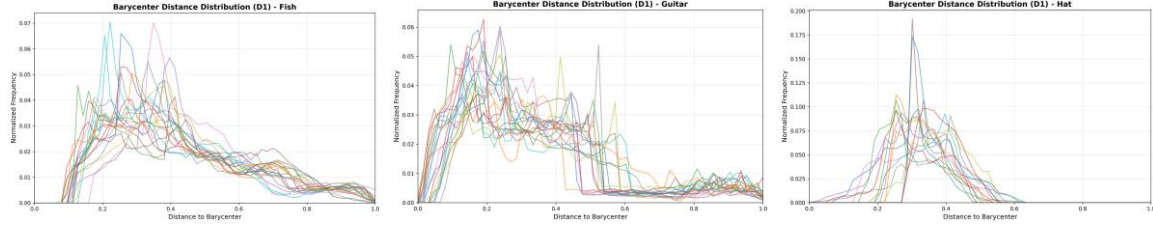


Figure 59: D1 descriptors for all shapes, grouped per class

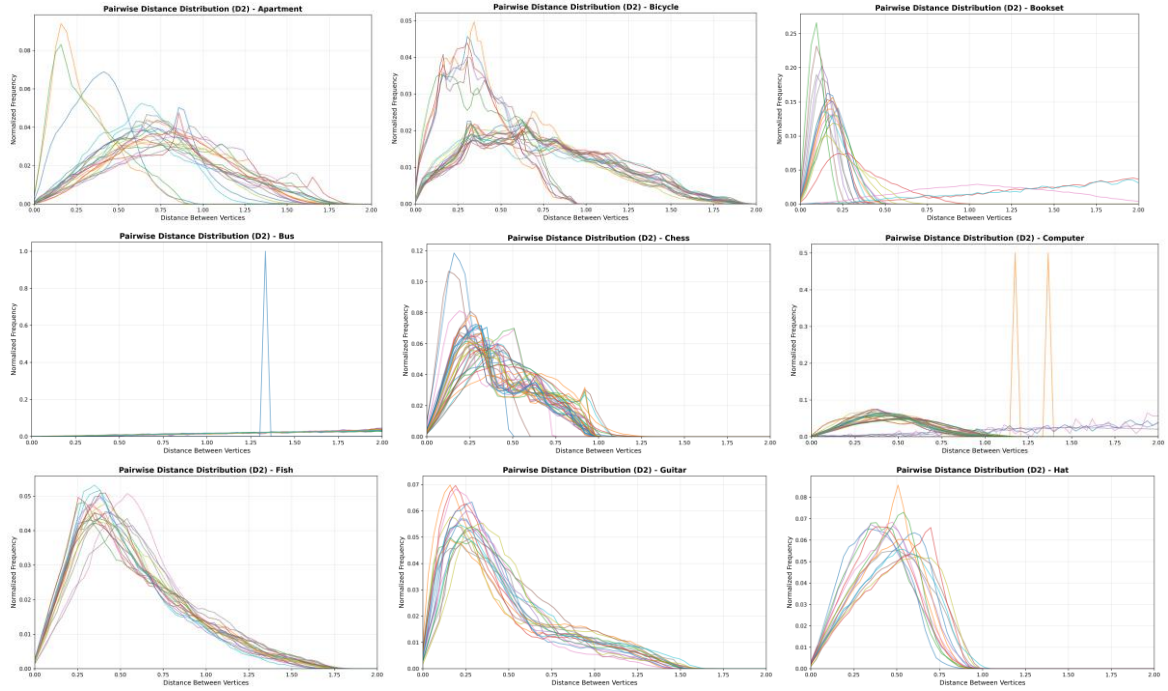
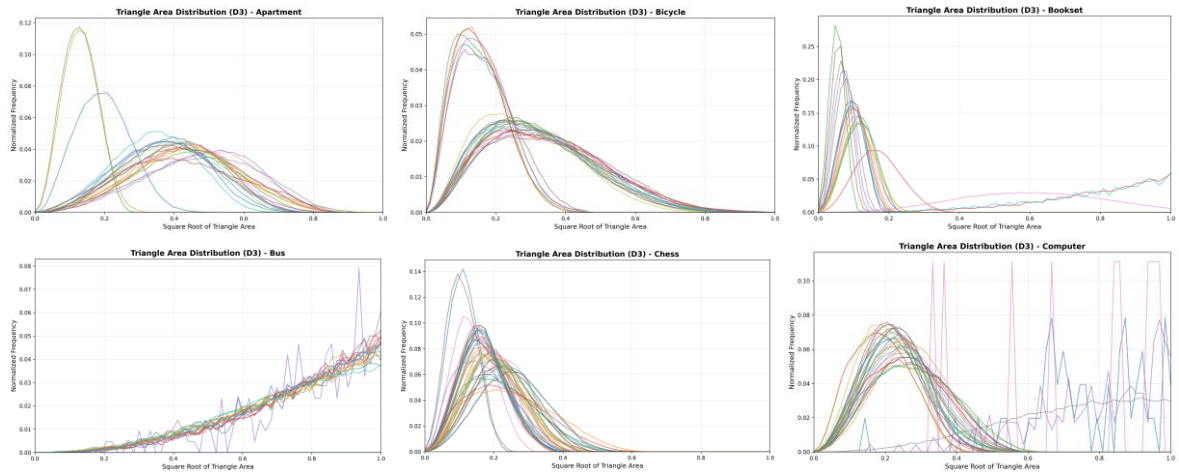


Figure 60: D2 descriptors for all shapes, grouped per class



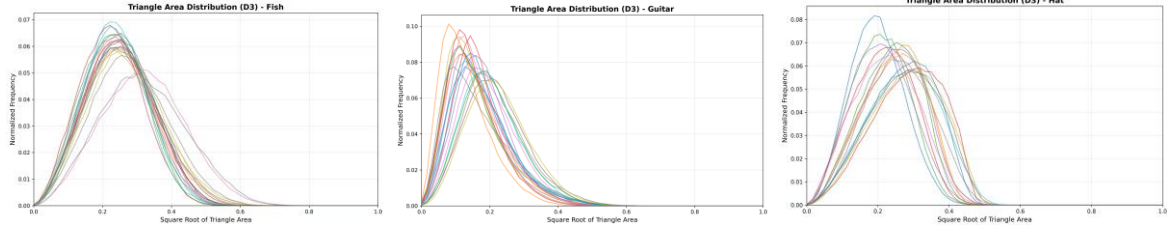


Figure 61: D3 descriptors for all shapes, grouped per class

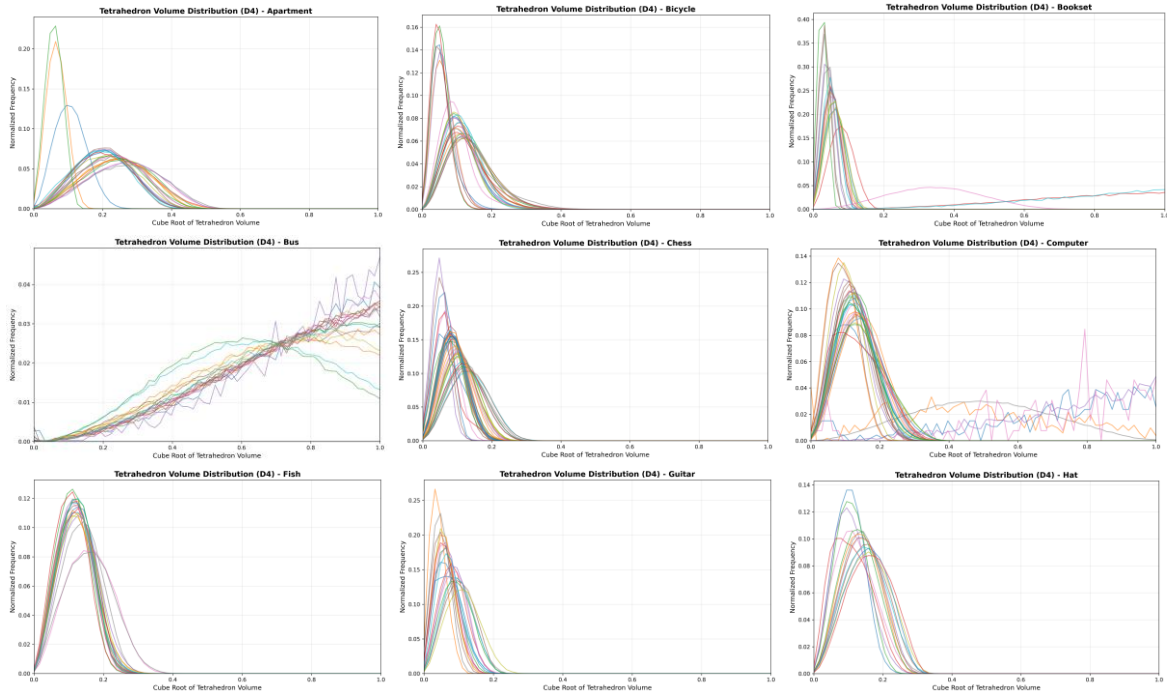


Figure 62: D4 descriptors for all shapes, grouped per class

6. Step 4 - Query

In this step, we implemented an end-to-end query system. When we give a query shape, the system extracts its features, computes distances to all shapes in the database, and finally displays the best matches to us.

6.1. Feature normalization-based query

Let $A = (a_1, \dots, a_n)$ be a feature vector. Typically A contains elements that describe different features, each having its own value. We have a feature vector of 507 elements. The first seven elements ($a_1 \dots a_7$) indicate elementary features, e.g., surface area, volume, compactness, rectangularity, diameter, convexity and eccentricity. Next, we have 100-bin histograms for the distribution descriptors A3 ($a_8 \dots a_{107}$), D1 ($a_{108} \dots a_{207}$), D2 ($a_{208} \dots a_{307}$), and D3 ($a_{308} \dots a_{407}$), and D4 ($a_{408} \dots a_{507}$).

To implement feature normalization-based queries, we first needed to normalize the features, so that they can be compared between different classes. For the seven elementary features, we calculated the global mean and standard deviation for each single value feature, and used the Z-score formula (shown below) to standardize the single value feature values. This normalized database is then saved. At the same time, the normalization parameters (global mean, std) are saved to a file so that the same parameters can be used to normalize the features of querying objects during queries. Because distribution features have already been normalized to probability distributions (sum to one) during feature extraction, they are not processed further in this step. The resulting normalized feature database and normalization parameter file are used later in the query code. Figure 63 shows parts of the merged feature files, normalized database, and normalization parameters. In the normalized chart, there are positive and negative values, indicating the success of feature normalization.

$$\text{Z-score formula: } z = \frac{(x - \mu)}{\sigma}$$

class_name ...	file_name (2)	surface_area (3)	volume (4)	compactness (5)	rectangularity (6)	diameter (7)	convexity (8)	eccentricity (9)
Bird	000113_full_norm.obj	7.205968856282875	0.2643099780778393	47.35847159108612	0.1215285948113528	1.76815000934245	0.2978056676535798	3.256486699963961
Bird	000164_full_norm.obj	3.762736822824128	0.0303339224030825	511.2455230589971	0.0316868050116944	1.9011206762291852	0.1064920547870199	6.792172710002124
Bird	000234_full_norm.obj	1.421013598965776	0.053347821781599	15.233483656741637	0.0437477940335447	1.8450749641630717	0.1730435344827337	5.4442559516029565
Bird	000271_full_norm.obj	2.2532285947975583	0.0212635892764463	223.71249368044255	0.0214103394136613	1.919004502823776	0.0653002557972554	12.469448177561414
Bird	000284_full_norm.obj	1.6363357998145065	0.0230315221147305	75.03308416062411	0.0265209222111511	1.9140362822890105	0.0841353285250708	4.42033698237189
Bird	000323_full_norm.obj	0.5829804946030749	0.0036645951108745	138.3118136104773	0.0461965660740966	1.9771382175127237	0.145787617394875	298.13531059986565
Bird	000386_full_norm.obj	1.2228355241245232	0.0202638250049032	39.373943533024566	0.096525323089174	1.8609589349444024	0.2213051303866771	135.21927584659232
Bird	000438_full_norm.obj	1.3168432399059442	0.01210110896023	137.87940320758256	0.0273470605479873	1.9511374257656744	0.1108480764366675	47.86061523175879
Bird	000442_full_norm.obj	3.6940843845113656	0.0481343632176563	192.37897238329467	0.0257890674203285	1.7882335286422688	0.0994946942949474	2.964283327647751
Bird	000545_full_norm.obj	1.222629979219662	0.0260970836367282	23.72732581468468	0.0610168526594613	1.896719339899324	0.2005653969646372	13.602357838395216
Bird	000599_full_norm.obj	0.977524504956545	0.007777831025656	136.52543806120966	0.0282582872013663	1.931653414808782	0.0999546175153017	59.72190105332561
Bird	000625_full_norm.obj	1.559128601839029	0.003553384254234	2651.066095210803	0.0014327577017029	1.89971762638558	0.0065772906701249	1.987460532866488
Bird	000689_full_norm.obj	1.2201404352214016	0.0072484198723363	305.6964541812101	0.0135408006108768	1.930154454182218	0.0418562434407643	10.023701607354688
Bird	000738_full_norm.obj	1.398534590325005	0.0172750211628354	81.04572067141928	0.034653031961059	1.971508920505309	0.1247713978433589	49.15479929154777
Bird	000762_full_norm.obj	1.7682968993295696	0.0638713888119616	11.983963116962826	0.0444184764151054	1.760217780894885	0.1733182324515463	3.816342322998456
Bird	000862_full_norm.obj	1.7686437077555044	0.0324554219567429	46.44020914766831	0.0366029772775896	1.920195638784228	0.1373601388766854	10.479209935488266

class_name	file_name (2)	surface_area (3)	volume (4)	compactness (5)	rectangularity (6)	diameter (7)	convexity (8)	eccentricity (9)
class_name	file_name	surface_area	volume	compactness	rectangularity	diameter	convexity	eccentricity
Apartment	D00045_Full_norm.obj	0.826617562834585207	-0.0681051145480511	0.018573876147987663	-0.0832762179372423	0.5950312755318985	-1.8390976711025672	-0.26643833717019777
Apartment	D00086_Full_norm.obj	0.804784616477871767	-0.023796805885746283	-0.07141940248158514	0.97197885515343	0.5237726230422252	1.5698064018771047	-0.22730849351798166
Apartment	D00138_Full_norm.obj	0.89337511449707327	-0.06013586299339744	-0.05832243571459644	-0.22252016188367663	0.6659210760523597	-0.6227853960818895	-0.25389330261848206
Apartment	D00156_Full_norm.obj	0.5392446196437344	-0.054255585877818217	0.11538800460103522	-0.38476417682527986	0.6682457916091122	-0.710915315152015	-0.24753154867334784
Apartment	D00310_Full_norm.obj	0.19668032661482937	-0.06186184158125585	-0.01679706134910471	-0.25232656018081867	0.6695280387970838	-0.6656257904804037	-0.23549752895618392
Apartment	D00348_Full_norm.obj	-0.823831170158906356	-0.03239335411676216	-0.07144010427909091	1.3448038254269416	0.4444470153031161	1.0217789334656442	-0.11503724768076202
Apartment	D00366_Full_norm.obj	0.825251445487502133	-0.08861617203498554	-0.07142768110473906	2.2355525834623835	0.5889438990331893	1.3549415674966552	-0.2630653520222634
Apartment	D00678_Full_norm.obj	0.84808208108641033	-0.06809373772210837	-0.07047965690347874	0.148621737577021	0.5623737484201222	0.29858448091673717	-0.2581859540549481
Apartment	D00696_Full_norm.obj	-0.817261642478132948	-0.02115762618448864	-0.07147411527910669	2.2961976480000818	0.45924995284866504	1.701026294847787	-0.267638744492813
Apartment	D00626_Full_norm.obj	0.88839041932724847	-0.04095620162342863	-0.07024127218172838	1.0627658027394657	0.5748465986770231	0.6624272076229718	-0.25297288364821213

feature (1)	mean (2)	std (3)
feature	mean	std
1		
2		
3		
4		
5		
6		
7		
8		

Figure 63: Parts of the merged feature files(top), normalized database(middle), and normalization parameters(bottom)

Next, we used the resampling, normalization, and feature extraction functions defined in the previous code to resample, normalize, and extract features from the input model file. Then, we used pre-saved normalization parameters to perform Z-score normalization on the single value features, ensuring that the query features and database features are on the same scale. Distribution features were not processed further as they were already normalized to probability distributions during extraction. Finally, we returned a feature vector of 507 elements for subsequent distance calculation with the database.

For two vectors, when calculating the distance between the query features and each model features in the database, we first chose to use Euclidean distance. We calculated the Euclidean distance once for the 7-dimensional single value features ($a_1..a_7$) and five times for the five groups (A3, D1, D2, D3, D4) of 100-dimensional distribution features. Finally, we summed all six distances to obtain the total distance. The function sorted the meshes by total distance from smallest to largest and returned the 10 most similar models.

For 2 combined features:

$$d_{Euclidean}(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

$$d_{total} = d_{single} + \sum_{k=1}^5 d_{distribution}$$

```

results for data/AquaticAnimal/m53.obj:
=====
1. Drum      | D00383_full_norm.obj | distance: 5.0792
2. Hand      | D01162_full_norm.obj | distance: 5.5428
3. Hand      | m332_full_norm.obj   | distance: 5.5428
4. House     | m462_full_norm.obj   | distance: 5.7054
5. Chess     | D01053_full_norm.obj | distance: 5.7557
6. Chess     | m1605_full_norm.obj  | distance: 5.7664
7. Hand      | m333_full_norm.obj   | distance: 6.1188
8. Drum      | D00115_full_norm.obj | distance: 6.3367
9. AquaticAnimal | m58_full_norm.obj   | distance: 6.6455
10. AquaticAnimal | m63_full_norm.obj   | distance: 6.7716

Top-10 class: ['Drum', 'Hand', 'Hand', 'House', 'Chess', 'Chess', 'Hand', 'Drum', 'AquaticAnimal', 'AquaticAnimal']

results for data/Guitar/D00023.obj:
=====
1. House     | m462_full_norm.obj   | distance: 5.0375
2. Chess     | D01053_full_norm.obj | distance: 5.0715
3. Chess     | m1605_full_norm.obj  | distance: 5.0779
4. Drum      | D00383_full_norm.obj | distance: 5.1065
5. Hand      | D01162_full_norm.obj | distance: 5.2535
6. Hand      | m332_full_norm.obj   | distance: 5.2539
7. Hand      | m333_full_norm.obj   | distance: 5.5744
8. Drum      | D00115_full_norm.obj | distance: 5.5928
9. AquaticAnimal | m58_full_norm.obj   | distance: 5.9455
10. AquaticAnimal | m63_full_norm.obj   | distance: 6.3569

Top-10 class: ['House', 'Chess', 'Chess', 'Drum', 'Hand', 'Hand', 'Hand', 'Drum', 'AquaticAnimal', 'AquaticAnimal']

```

Figure 64: Query results for AquaticAnimal/m55.obj and Guitar/D00023.obj based on Euclidean distance

Since we queried several objects(AquaticAnimal/m55.obj and Guitar/D00023.obj), we found the results are terrible, which are shown in Figure 64. So instead of using the Euclidean distance, we tried to use the cosine distance, which is the complement of the cosine similarity(formula shown in below), to calculate the distance between two feature vectors. Figure 65 shows the query results, but they are also terrible. The poor performance is likely due to inconsistent distance ranges. Because our mesh quality is unstable during feature extraction. The distance ranges between different features varies greatly, and the distance ranges of the same feature (e.g., surface_area) can differ significantly between different classes. Therefore, the current query cannot effectively balance the contribution of each feature in similarity calculation. It leads to some features dominating the query results while others are ignored. Therefore, our retrieval failed. To address this issue, we tried to use different method to measure distances between different features and give different weights to different features distances.

$$\text{COS}_{similarity}(x,y) = \frac{x \cdot y}{|x| \cdot |y|}$$

```

results for data/AquaticAnimal/m53.obj:
=====
1. Guitar      | D00232_full_norm.obj      | distance: 1.1253
2. Glasses    | D00994_full_norm.obj      | distance: 1.1492
3. Gun        | m651_full_norm.obj        | distance: 1.1514
4. Gun        | D00802_full_norm.obj      | distance: 1.1738
5. Gun        | m662_full_norm.obj        | distance: 1.1846
6. Guitar     | D00464_full_norm.obj      | distance: 1.1875
7. Glasses    | D00630_full_norm.obj      | distance: 1.2012
8. Guitar     | D00222_full_norm.obj      | distance: 1.2020
9. Gun        | D00207_full_norm.obj      | distance: 1.2112
10. Bird      | D00736_full_norm.obj      | distance: 1.2129

Top-10 class: ['Guitar', 'Glasses', 'Gun', 'Gun', 'Gun', 'Guitar', 'Glasses', 'Guitar', 'Gun', 'Bird']

results for data/Guitar/D00023.obj:
=====
1. Gun        | D00477_full_norm.obj      | distance: 0.8978
2. Gun        | D00826_full_norm.obj      | distance: 1.0116
3. Gun        | D00282_full_norm.obj      | distance: 1.0440
4. Bird       | D00386_full_norm.obj      | distance: 1.1491
5. Guitar     | D00461_full_norm.obj      | distance: 1.1957
6. Gun        | D00153_full_norm.obj      | distance: 1.2503
7. Gun        | D00802_full_norm.obj      | distance: 1.3239
8. Guitar     | D00549_full_norm.obj      | distance: 1.3380
9. Guitar     | D00374_full_norm.obj      | distance: 1.3419
10. Gun       | D00137_full_norm.obj      | distance: 1.3478

Top-10 class: ['Gun', 'Gun', 'Gun', 'Bird', 'Guitar', 'Gun', 'Gun', 'Guitar', 'Guitar', 'Gun']

```

Figure 65: Query results for AquaticAnimal/m53.obj and Guitar/D00023.obj based on cosine distance

6.2. Query improvement

To improve the query performance, we redesigned the similarity computation by using different distance measures to the two groups of features. For the seven single-value descriptors ($a_1, \dots, a_7$), we computed the feature distance dimension-wise using the L1 metric:

$$d_{elementary, i} = |a_i^{(q)} - a_i^{(m)}|, i = 1, \dots, 7$$

Where $a_i^{(q)}$ and $a_i^{(m)}$ denote the i -th feature of the query mesh and a database mesh, respectively. Each of these distances is scaled by a weight w_i , producing the weighted single-value distance:

$$d_{SV} = \sum_{i=1}^7 w_i d_{elementary, i}$$

For the five distribution descriptors, Euclidean or cosine distance is not appropriate because it does not capture the geometric structure of probability distributions. Therefore, we used the Earth Mover's Distance (EMD), implemented using the 1D Wasserstein distance, to measure the difference between two histograms:

$$d_{EMD, k} = W_1(H_k^{(q)}, H_k^{(m)}), k = 1, \dots, 5$$

where H_k denotes the k -th 100-bin distribution (A3, D1, D2, D3, D4). Each histogram distance is also weighted:

$$d_{HD} = \sum_{k=1}^5 \alpha_k d_{EMD,k}$$

Finally, the total similarity distance between the query mesh and a database mesh is given by the weighted sum of single-value and histogram distances:

$$d_{total} = d_{SV} + d_{HD}$$

```
WEIGHTS = {
  'single_value': {
    'surface_area': 0.015,
    'volume': 0.005,
    'compactness': 0.010,
    'rectangularity': 0.025,
    'diameter': 0.025,
    'convexity': 0.010,
    'eccentricity': 0.010,
  },
  'histogram': {
    'A3': 0.3,
    'D1': 0.05,
    'D2': 0.21,
    'D3': 0.22,
    'D4': 0.22,
  }
}
```

Figure 66: Weight list for different features

Figure 66 shows the weight list for different features. Initially, we discovered that due to mesh quality issues, single-valued descriptors were prone to extreme values. So we wanted to reduce their weights, especially for volume, since the technical tips didn't require them to be included as a feature vector. Therefore, we gave single-valued descriptors a very low weight. As for distribution descriptors, we mainly assigned weights based on the quality of their histograms. For example, the D1 histogram was less clustered than other histograms, so I gave it a lower weight. Besides, these weights can be gradually adjusted in the future based on the evaluation results.

```
Results for data/AquaticAnimal/m53.obj:
=====
1. AquaticAnimal | m53_simplified_norm.obj | distance: 0.0003
2. AquaticAnimal | m68_refined_norm.obj | distance: 0.0110
3. Humanoid | m192_refined_norm.obj | distance: 0.0128
4. Rocket | D00346_refined_norm.obj | distance: 0.0132
5. Humanoid | m160_simplified_norm.obj | distance: 0.0166
6. Humanoid | m119_refined_norm.obj | distance: 0.0169
7. Humanoid | m124_refined_norm.obj | distance: 0.0175
8. Ship | m1465_kept_norm.obj | distance: 0.0176
9. AquaticAnimal | m57_simplified_norm.obj | distance: 0.0181
10. Rocket | D00719_kept_norm.obj | distance: 0.0195

Top-10 classes: ['AquaticAnimal', 'AquaticAnimal', 'Humanoid', 'Rocket', 'Humanoid', 'Humanoid', 'Humanoid', 'Ship', 'AquaticAnimal', 'Rocket']

Results for data/Guitar/D00023.obj:
=====
1. Guitar | D00023_simplified_norm.obj | distance: 0.0008
2. Musical_Instrument | m625_simplified_norm.obj | distance: 0.0031
3. Knife | D01120_refined_norm.obj | distance: 0.0186
4. Musical_Instrument | m632_simplified_norm.obj | distance: 0.0259
5. Guitar | D00645_simplified_norm.obj | distance: 0.0284
6. Train | D01016_simplified_norm.obj | distance: 0.0302
7. Door | D01136_simplified_norm.obj | distance: 0.0336
8. Gun | m671_simplified_norm.obj | distance: 0.0338
9. Knife | m728_refined_norm.obj | distance: 0.0360
10. Humanoid | m189_simplified_norm.obj | distance: 0.0364

Top-10 classes: ['Guitar', 'Musical_Instrument', 'Knife', 'Musical_Instrument', 'Guitar', 'Train', 'Door', 'Gun', 'Knife', 'Humanoid']
```

Figure 67: Query results for AquaticAnimal/m55.obj and Guitar/D00023.obj based on L1 and EMD

After adopting the new strategy, we queried again for the meshes(AquaticAnimal/m55.obj and Guitar/D00023.obj) that had been queried using the other two methods. Figure 67 shows the results. In this query, both meshes retrieved similar meshes, demonstrating significantly better performance than the previous two methods.

To improve the query results later, we tried to select ten classes and remove some bad meshes from them, and used the ten classes to test querying. This removal process was based on calculated feature values. Outliers that significantly deviate from other feature values were removed. Meshes with such outliers are usually visually weird. However, judging mesh quality solely by visual inspection is not very accurate. Figure 68 shows two meshes with outliers (marked in red in the table). One mesh is visually identifiable as having poor quality, while the other is not.

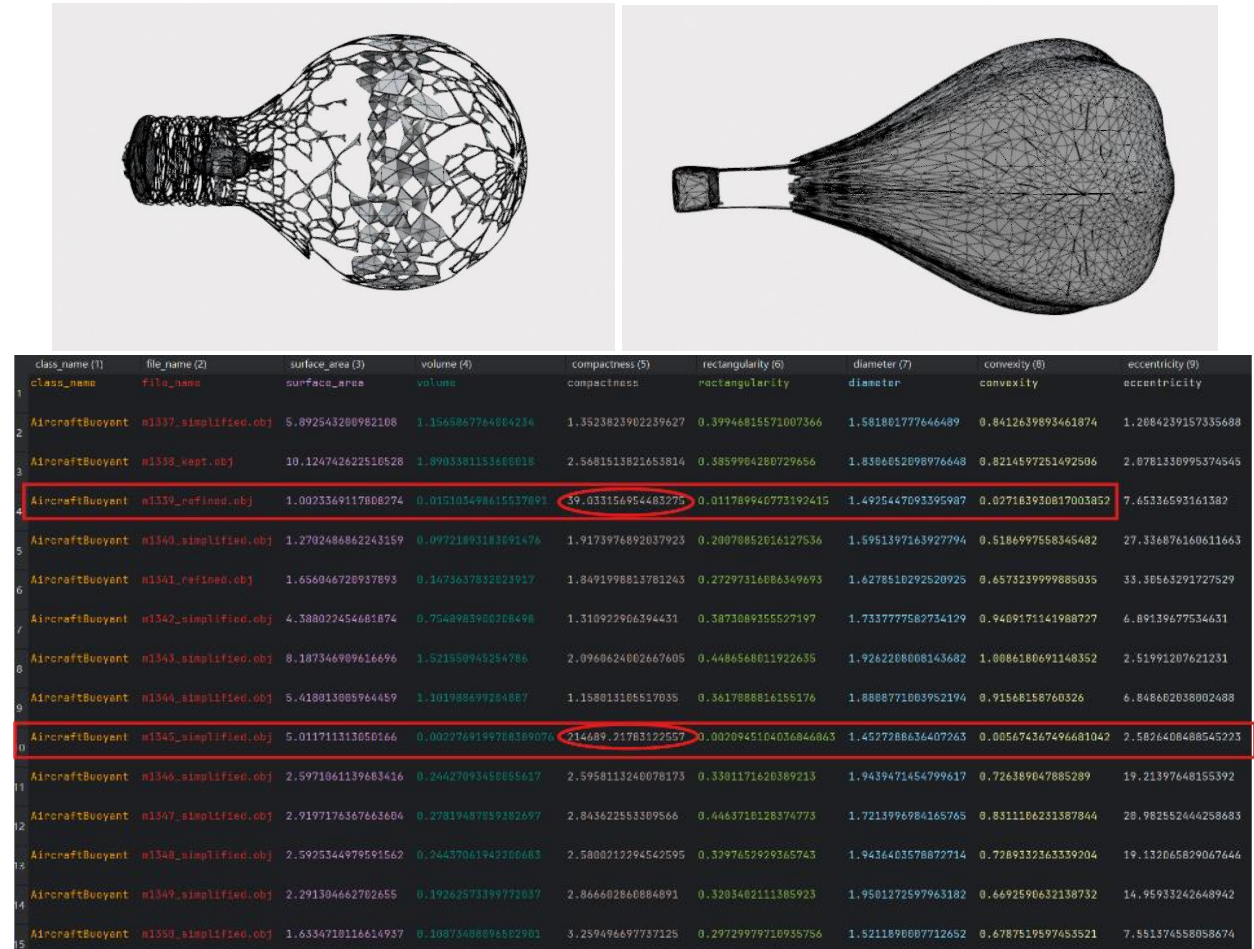


Figure 68: Two meshes with outliers, and their outlier value

Finally, we wanted to obtain classes that are close to having no outliers, and their histograms are more concentrated. Figure 69 shows the histogram before mesh removal, and Figure 70 shows the histogram after mesh removal, it is obviously more concentrated.

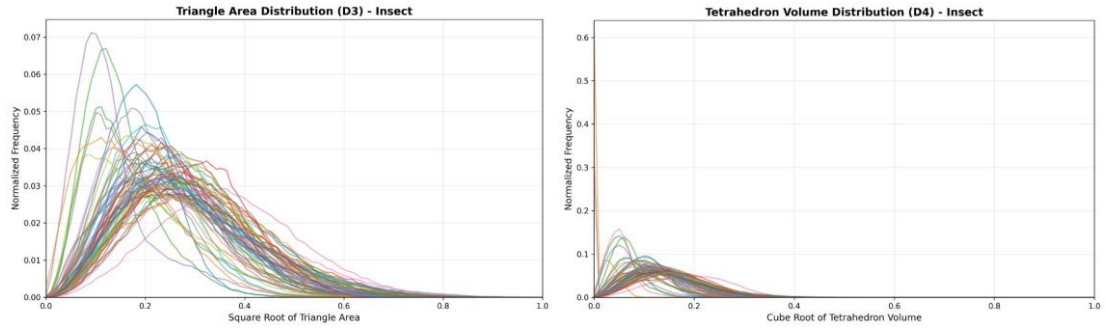


Figure 69: D3, D4 histograms of Insect classes before mesh removal

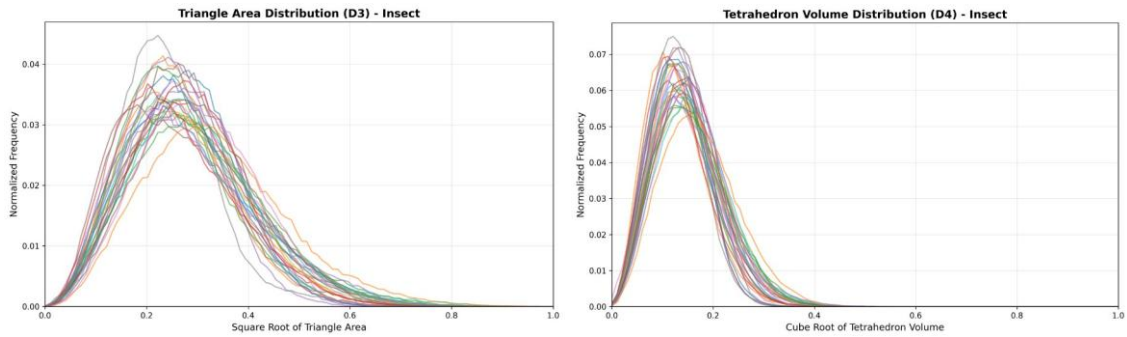


Figure 70: D3, D4 histograms of Insect classes after mesh removal

```
Results for data/Bicycle/D00536.obj:
=====
1. Bicycle      | D00536_simplified_norm.obj | distance: 0.0096
2. Bicycle      | D00847_simplified_norm.obj | distance: 0.0138
3. Bicycle      | D00078_simplified_norm.obj | distance: 0.0159
4. Bicycle      | D00555_simplified_norm.obj | distance: 0.0210
5. Bicycle      | D00550_simplified_norm.obj | distance: 0.0289
6. Bicycle      | D00930_simplified_norm.obj | distance: 0.0300
7. Bicycle      | m1477_simplified_norm.obj  | distance: 0.0351
8. Bicycle      | D00750_simplified_norm.obj | distance: 0.0354
9. Bicycle      | D00931_simplified_norm.obj | distance: 0.0384
10. Bicycle     | m1472_refined_norm.obj     | distance: 0.0391

Top-10 classes: ['Bicycle', 'Bicycle', 'Bicycle', 'Bicycle', 'Bicycle', 'Bicycle', 'Bicycle', 'Bicycle', 'Bicycle', 'Bicycle']

Results for data/Helicopter/D00397.obj:
=====
1. Helicopter   | D00397_kept_norm.obj       | distance: 0.0017
2. Helicopter   | D00977_kept_norm.obj       | distance: 0.0093
3. Helicopter   | m1326_simplified_norm.obj   | distance: 0.0167
4. Helicopter   | m1323_kept_norm.obj        | distance: 0.0171
5. Helicopter   | m1328_simplified_norm.obj   | distance: 0.0172
6. Helicopter   | m1325_simplified_norm.obj   | distance: 0.0209
7. Helicopter   | D00191_simplified_norm.obj  | distance: 0.0220
8. Helicopter   | D00658_simplified_norm.obj  | distance: 0.0272
9. Helicopter   | D00498_refined_norm.obj     | distance: 0.0360
10. Helicopter  | m1302_simplified_norm.obj   | distance: 0.0371

Top-10 classes: ['Helicopter', 'Helicopter', 'Helicopter', 'Helicopter', 'Helicopter', 'Helicopter', 'Helicopter', 'Helicopter', 'Helicopter', 'Helicopter']
```

```

Results for data/Knife/D01054.obj:
=====
1. Knife      | D01054_refined_norm.obj      | distance: 0.1751
2. Knife      | D01174_simplified_norm.obj    | distance: 0.3620
3. Knife      | D01075_simplified_norm.obj    | distance: 0.4369
4. Knife      | D01168_simplified_norm.obj    | distance: 0.4478
5. Knife      | D01119_refined_norm.obj      | distance: 0.4481
6. Knife      | D01051_refined_norm.obj      | distance: 0.4510
7. Knife      | D01057_simplified_norm.obj    | distance: 0.4599
8. Knife      | D01160_simplified_norm.obj    | distance: 0.4650
9. Knife      | D01077_simplified_norm.obj    | distance: 0.4833
10. Knife     | D01120_refined_norm.obj      | distance: 0.4890

Top-10 classes: ['Knife', 'Knife', 'Knife', 'Knife', 'Knife', 'Knife', 'Knife', 'Knife', 'Knife', 'Knife']

Results for data/Fish/D00024.obj:
=====
1. Fish       | D00024_simplified_norm.obj    | distance: 0.0018
2. Fish       | D00165_simplified_norm.obj    | distance: 0.0131
3. Fish       | D00030_simplified_norm.obj    | distance: 0.0152
4. Fish       | D00022_simplified_norm.obj    | distance: 0.0304
5. Fish       | D00012_simplified_norm.obj    | distance: 0.0353
6. Fish       | D00535_simplified_norm.obj    | distance: 0.0397
7. Fish       | D00643_simplified_norm.obj    | distance: 0.0450
8. Fish       | D00850_simplified_norm.obj    | distance: 0.0499
9. Fish       | D00682_simplified_norm.obj    | distance: 0.0508
10. Fish      | D00398_simplified_norm.obj    | distance: 0.0525

Top-10 classes: ['Fish', 'Fish', 'Fish', 'Fish', 'Fish', 'Fish', 'Fish', 'Fish', 'Fish', 'Fish']

```

Figure 71: 4 query results of meshes in different classes after mesh removal

After removing the bad meshes, we queried four different types of meshes, and each time the returned results were all meshes of the same type (shown in Figure 71), which proves that removing bad meshes can significantly improve query quality.

6. Step 5: Scalability

The aim of this step was to make the Content-Based Shape Retrieval (CBSR) system faster and more practical for large-scale databases.

In earlier stages, each query compared the input shape with every other shape in the dataset, which ensured correctness but resulted in slow performance.

Step 5 focuses on improving this by introducing (1) an efficient *k-nearest-neighbour* (k-NN) search engine and (2) a *t-SNE* visualization that reveals how the extracted features are organized in the feature space, these additions make the system both scalable and interpretable.

6.1. Data Preparation and Feature Loading

The process begins by loading the feature database generated in Step 3.

Each entry represents a 3D shape described by a numerical feature vector and its corresponding class label.

The loading function automatically checks for missing information and infers the file name or class name if necessary.

All features are standardised using the `StandardScaler` to ensure equal contribution of every dimension during distance computation.

This preprocessing step guarantees consistent feature representation before applying the neighbour search algorithms.

6.2. Efficient Neighbour Search

6.2.1. Exact k-Nearest Neighbours

The first retrieval method used Scikit-learn's `NearestNeighbors` class with Euclidean distance.

This *exact* approach computes true nearest neighbours for each query and provides a baseline for correctness.

Although accurate, it scales linearly with the dataset size and thus becomes inefficient for large collections.

6.2.2. Approximate Nearest Neighbours (ANN)

To overcome this limitation, an approximate method was implemented using the **Annoy** library.

This method builds a spatial index using multiple random projection trees, which enables very fast retrieval while maintaining near-identical accuracy.

The trade-off between exactness and speed is minimal, and for larger datasets the performance improvement is significant.

The benchmark results in Figure 36 show the average query time for various K values. While the exact method becomes progressively slower, the ANN engine maintains almost constant query time even as K increases. This demonstrates that the approximate index successfully scales the CBSR system to larger databases.

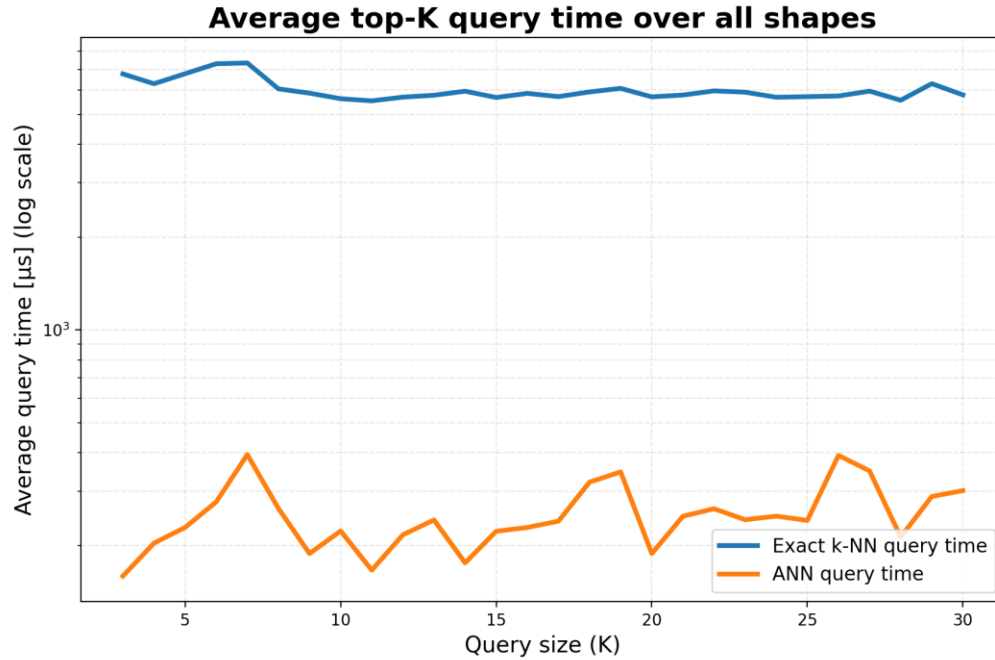


Figure 35: Average query time (log scale) comparing exact and approximate k -NN search. The approximate engine remains nearly constant as K grows, confirming improved scalability.

6.3. Feature Space Visualization with t-SNE

Improving speed alone is not enough; it is also important to understand how well the extracted features distinguish different shape categories.

For this purpose, *t-Distributed Stochastic Neighbor Embedding* (t-SNE) was used to project the high-dimensional features into a two-dimensional plane.

Each point in the plot represents a shape, and the colour corresponds to its class label. The resulting visualization, shown in Figure 37, highlights clear clusters for distinctive categories such as *Bicycle* or *Biplane*, while geometrically similar classes such as *Table* and *Door* tend to overlap.

This indicates that the feature extraction captures essential structural differences but may confuse objects with similar geometry.

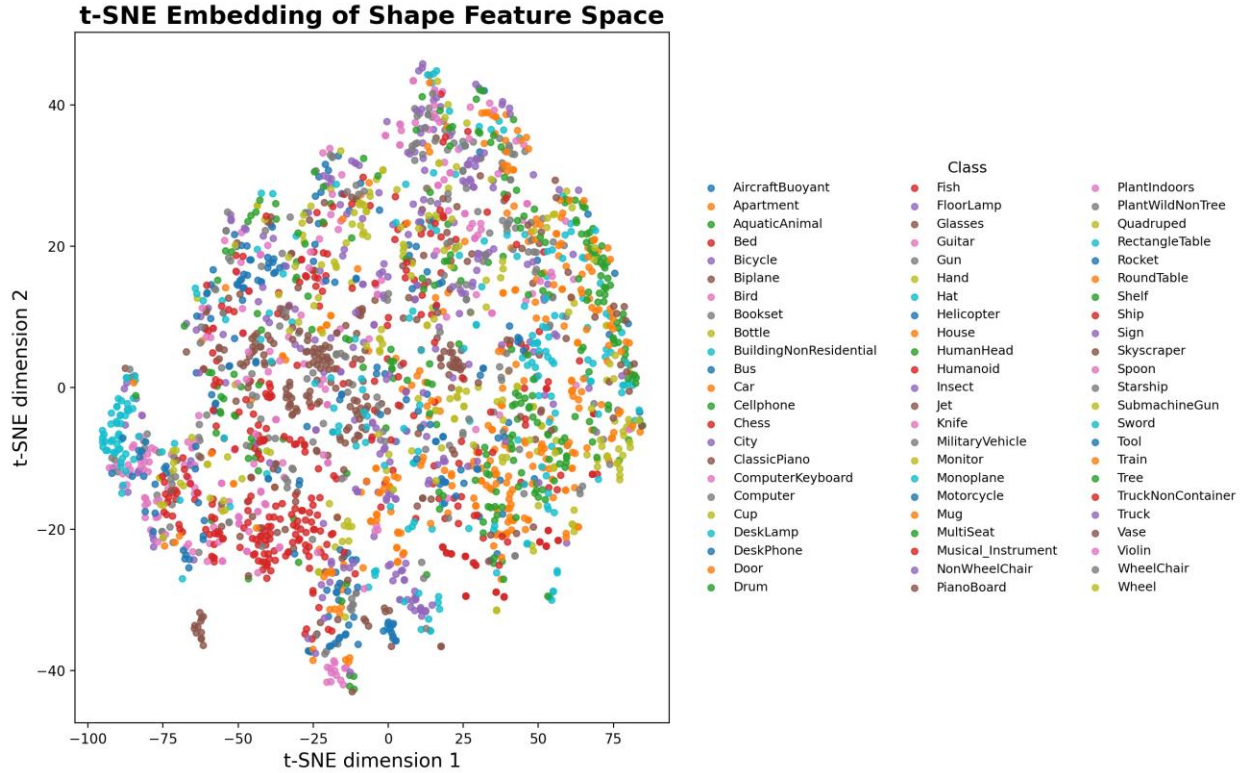


Figure 36: Two-dimensional *t*-SNE projection of all shapes, coloured by class. Compact single-colour clusters show strong class separation, while mixed colours indicate overlapping features.

6.4. Interactive Retrieval Interface

To demonstrate the scalability improvements in a practical setting, a web-based interface was developed using **Streamlit**.

This interface allows users to upload a 3D model and instantly view the most similar shapes from the database. Each result is displayed as a 3D plot rendered through Plotly and arranged in a grid for visual comparison.

The interface uses the same k-NN retrieval logic as the backend script, making the experiment fully interactive.

Figure 38 presents an example of the application. The top panel shows the uploaded query mesh, and the lower section displays the ten retrieved shapes ranked by similarity. The high visual resemblance among the retrieved shapes confirms that the system identifies meaningful geometric similarities.

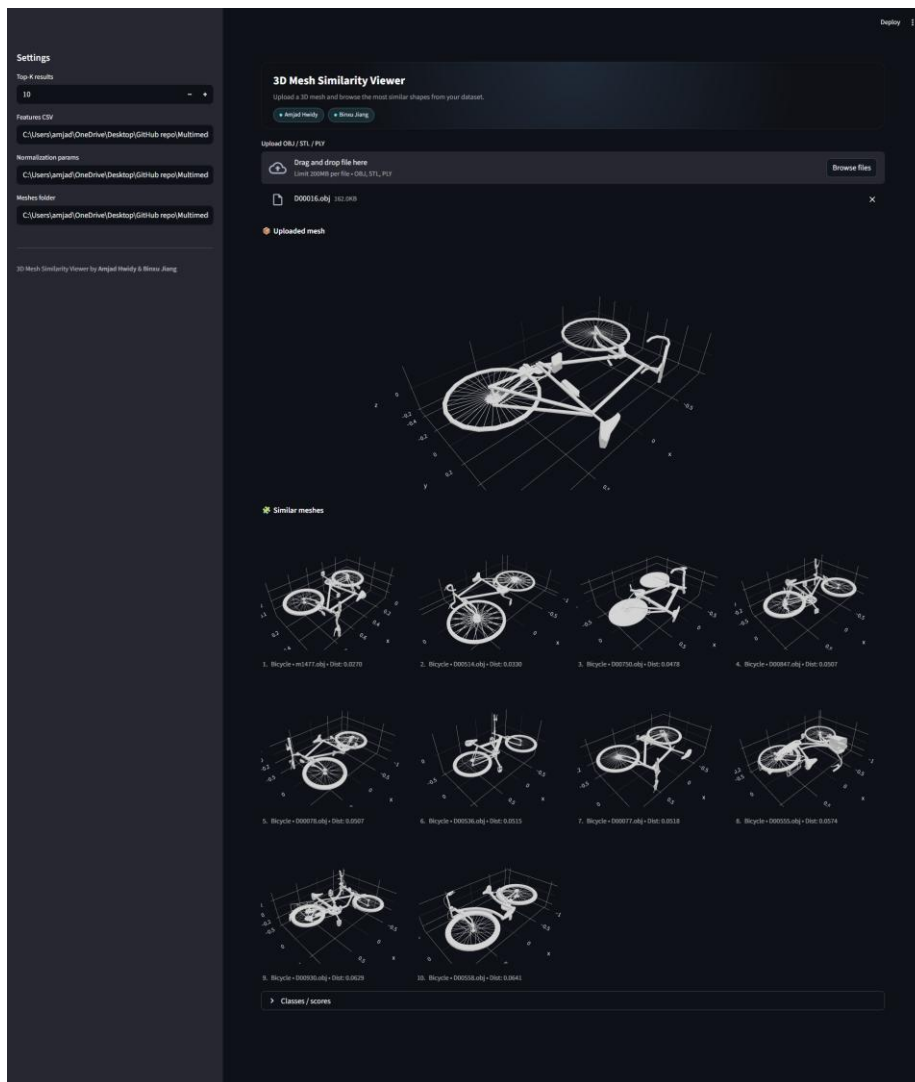


Figure 37: Streamlit-based 3D mesh retrieval interface. The uploaded query (top) retrieves the ten most similar meshes (bottom) using the feature-based k -NN search.

6.5. Conclusion of Step 5

Step 5 successfully transformed the CBSR system into a scalable and interpretable tool. The approximate neighbour search reduced query times dramatically while preserving retrieval quality, and the t-SNE visualization provided valuable insight into class separability.

The Streamlit interface further demonstrated that the system performs efficiently and intuitively even with larger shape datasets. This step thus established the foundation for evaluating retrieval accuracy in the next phase.

7.Step 6: Evaluation

After achieving scalability, the next goal was to assess how accurately the system retrieves shapes belonging to the same class. Step 6 provides a quantitative evaluation of the retrieval quality by comparing predicted neighbours with the true class labels. Each shape in the database serves once as a query, ensuring a complete and objective assessment.

7.1. Methodology

For every query shape, the system retrieves its K nearest neighbours (excluding itself) and records how many of them share the same class. From this comparison, four possible outcomes are determined:

- **True Positives (TP)** when retrieved shapes belong to the correct class.
- **False Positives (FP)** when they belong to other classes.
- **False Negatives (FN)** when relevant shapes were not retrieved.
- **True Negatives (TN)** for all other cases.

These values form the basis for calculating evaluation metrics such as precision, recall, sensitivity, and specificity.

Precision measures the proportion of correct results among the retrieved shapes, while Recall measures how many relevant shapes are successfully found.

Sensitivity and specificity further express the system's ability to retrieve relevant items while avoiding irrelevant ones.

The evaluation was repeated for several K values (1, 3, 5, 10, 20, 30) to understand how performance changes as more neighbours are included.

7.2. Quantitative Results

To analyse how retrieval quality changes with different values of K , the precision and recall scores were computed for both the exact and approximate (ANN) engines.

The following results present the two main experiments: one for the **dataset with all normalized classes** and another for the **dataset with the 10 cleaned classes** used for validation and visualization in the Streamlit interface.

7.2.1. Precision–Recall Analysis for All Normalized Classes

For the complete normalized dataset, the overall precision starts below 0.5, and recall values remain relatively low across K values.

This behaviour reflects the complexity of the full dataset, which contains many overlapping and visually similar classes.

The approximate engine (ANN) follows the same general trend as the exact search, confirming that the approximation does not significantly affect retrieval accuracy.

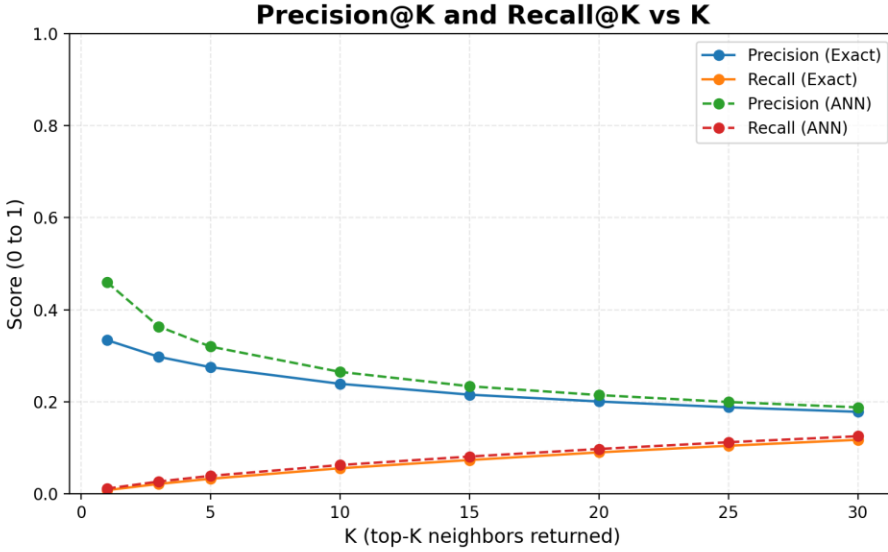


Figure 38: All normalized classes, Precision at K and Recall at K for different K values. Both search engines display the expected trade-off between precision and recall, showing nearly identical performance. Precision starts below 0.5 and gradually decreases as K increases, while recall slowly improves, indicating that including more neighbours increases completeness but slightly reduces accuracy.

7.2.2. Precision–Recall Analysis for 10 Cleaned Classes

For the smaller and cleaner subset of 10 classes, the precision and recall trends are much stronger and clearer. Precision starts near 0.8 for small K values, showing that the first retrieved shapes are highly relevant to the query.

As K increases, precision decreases moderately, while recall rises steadily, reflecting that the system retrieves more correct shapes as the neighbour window expands.

This demonstrates that the extracted features effectively separate the cleaned classes and that the ranking of neighbours behaves as expected.

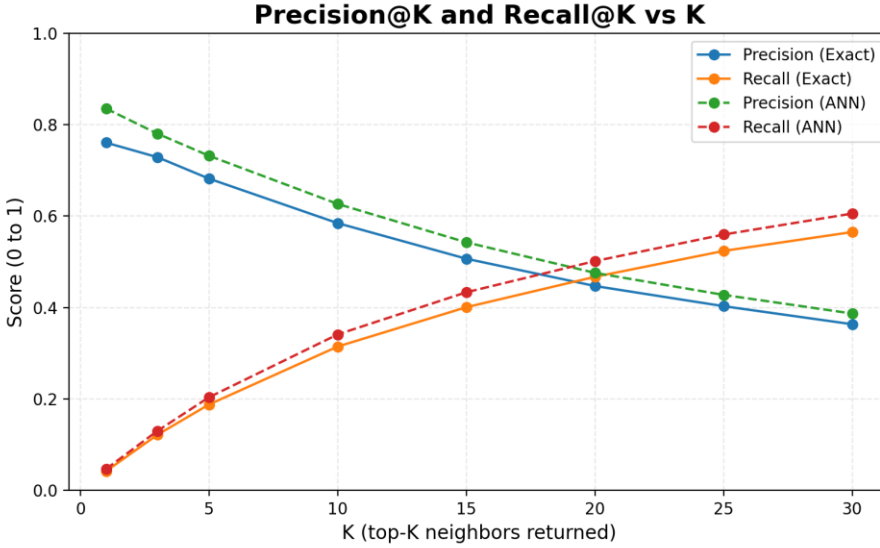


Figure 39: Ten cleaned classes, Precision at K and Recall at K for different K values. Both search engines display the expected trade-off and nearly identical performance. Precision starts around 0.8 for low K values, confirming strong early retrieval accuracy, while recall increases consistently as K grows.

7.2.3. Sensitivity–Specificity Analysis

To further evaluate the balance between correctly retrieved and rejected shapes, the sensitivity (true positive rate) and specificity (true negative rate) were analysed for both datasets.

This analysis provides a broader understanding of how the system performs when distinguishing between relevant and irrelevant items.

For the 10 cleaned classes, both sensitivity and specificity remain high across all K values, showing that the retrieval engine maintains a stable and accurate balance. As visible in Figure 41, the approximate (ANN) engine performs almost identically to the exact k-NN method, with both curves lying well above the random baseline.

This means that the approximate search achieves high recall without increasing the rate of false positives, proving that scalability does not compromise reliability.

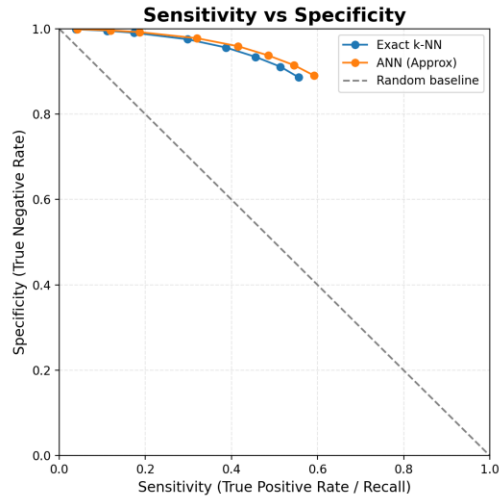


Figure 40: Sensitivity versus specificity for 10 cleaned classes. Both the exact and approximate search engines maintain a strong balance between sensitivity and specificity. The curves remain far above the random baseline, confirming that the system retrieves re

For the dataset containing all normalized classes, the performance remains high but slightly flatter near the top of the plot, as shown in Figure 42. This is expected given the higher-class diversity and overlap between geometrically similar categories.

Despite this added complexity, both engines still achieve high specificity and sensitivity, indicating that the system can handle heterogeneous datasets without a major loss in accuracy.

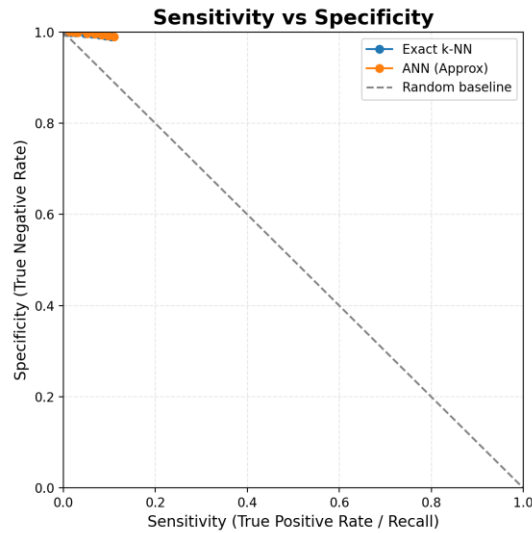


Figure 41: Sensitivity versus specificity for all normalized classes. Both search engines stay well above the random baseline. Although the curves flatten slightly due to increased class overlap, retrieval remains robust and stable for the entire dataset.

7.2.4. Retrieval Outcome Distribution

To understand the overall retrieval behaviour, the total numbers of **True Positives (TP)**, **False Positives (FP)**, **False Negatives (FN)**, and **True Negatives (TN)** were computed for $K = 30$.

These values show how many shapes were classified correctly or incorrectly across all queries.

For the 10 cleaned classes, shown in Figure 43, true negatives dominate, followed by a considerable number of true positives. Only a small proportion of false positives and false negatives occur, which means that the system retrieves the correct shapes for most queries and rarely confuses unrelated classes.

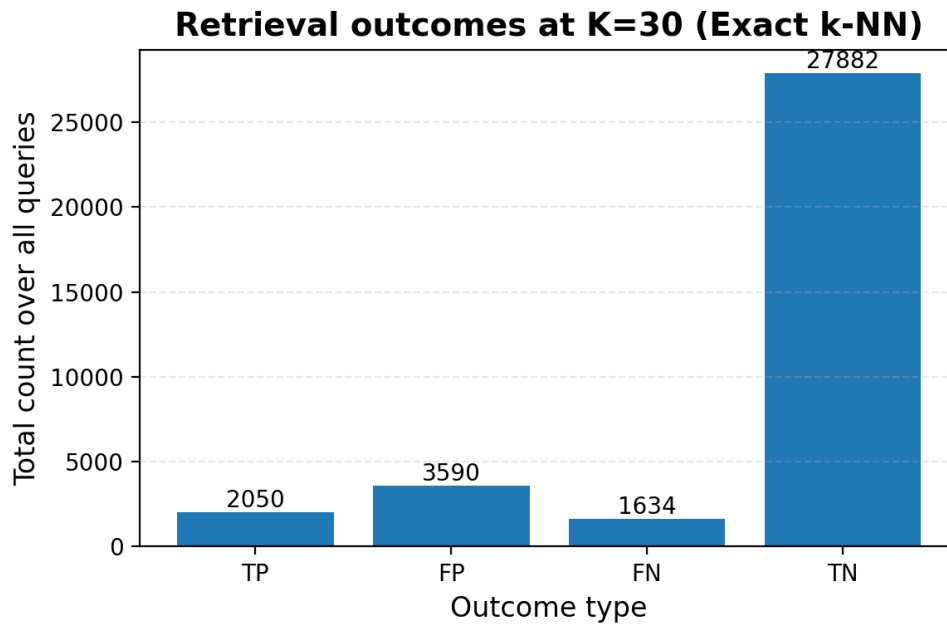


Figure 42: Retrieval outcomes at $K = 30$ for the 10 cleaned classes (Exact k-NN). True positives and true negatives represent the majority of outcomes, while incorrect predictions remain minimal, confirming the high retrieval reliability of the cleaned dataset.

In contrast, the results for the complete normalized dataset (Figure 44) reveal higher absolute counts due to its larger size and greater variety of shapes. Although the numbers of false positives and false negatives increase, true negatives still vastly outnumber the incorrect outcomes, which demonstrates that the system maintains stability even when the data distribution is more complex.

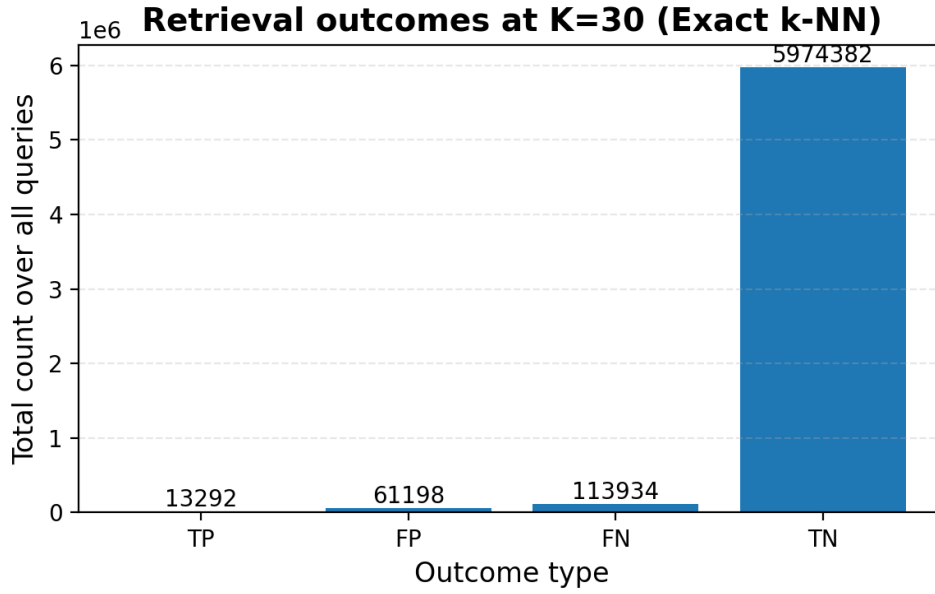


Figure 43: Retrieval outcomes at $K = 30$ for all normalized classes (Exact k -NN). True negatives clearly dominate, and despite the higher dataset complexity, the ratio between true and false outcomes confirms that the system performs reliably on a larger scale.

7.3. Discussion of Results

The comparative evaluation between the two datasets highlights how dataset structure affects retrieval accuracy. When the data are cleaned and well-defined, the system reaches a precision of about 0.8 for small K values and maintains a strong recall curve.

This indicates that the extracted geometric features effectively separate the classes and that the ranking mechanism is well-behaved. For the full dataset, which contains many overlapping categories, precision falls below 0.5, mainly because certain classes share similar structural characteristics.

However, the consistent precision–recall patterns and high specificity show that the retrieval model remains reliable and interpretable. Across all tests, the approximate (ANN) search engine achieves results almost identical to the exact k -NN method while operating much faster.

This confirms that the system’s scalability does not come at the cost of retrieval quality. The high sensitivity–specificity values, together with the dominance of true results in the outcome distributions, demonstrate that the CBSR system retrieves relevant shapes efficiently and accurately.

7.4. Conclusion of Step 6

Step 6 completes the quantitative evaluation of the scalable CBSR system. By analysing precision, recall, sensitivity, and specificity across different datasets, it demonstrates that the system performs reliably for both clean and complex data.

The approximate nearest-neighbour index delivers the same retrieval quality as the exact search while providing substantial computational benefits.

Overall, the results confirm that the final system can retrieve semantically correct 3D shapes with strong accuracy, clear interpretability, and high efficiency successfully meeting the objectives of this stage.

8. Summary

This project implements a complete end-to-end 3D shape retrieval system that integrates mesh resampling, normalization, multi-level feature extraction, feature standardization, and similarity-based querying.

The system first performs resampling and normalization on the input mesh to ensure a stable vertex count, unified scale, and aligned pose.

Based on the normalized meshes, two groups of features are extracted: seven elementary 3D shape descriptors and five distribution descriptors (A3, D1, D2, D3, and D4). During feature standardization, Z-score normalization is applied to the single-value descriptors to improve cross-class comparability, while distribution descriptors are preserved as probability histograms.

For similarity computation, L1 distance is used for the single-value features, whereas the Earth Mover's Distance (Wasserstein-1) is employed for the histogram-based descriptors. A weighted combination of these distances yields a unified similarity measure.

Finally, the system returns the meshes with the 10 smallest total distances to the query.

References

Murel, Jacob, PhD, and Meredith Syed. “Information Retrieval.” *Think*, 17 Apr. 2025, www.ibm.com/think/topics/information-retrieval.